UNIT - II

LINEAR DATA STRUCTURES, SEARCHING AND SORTING

CO2: Apply various sorting and searching algorithms for given problem.

2.1 OVERVIEW OF ARRAY

When we declare variables as:

```
a = b = c = 0  # or any default value you want
```

Compiler reserves 3 memory locations to store 3 integer numbers.

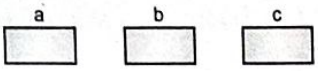


Fig. 2.1: Memory allocation for variables

In case, we want to store more numbers, say 1000 we have to have those many variables declared. But this is going to be inconvenient for the programmer.

C provides a convenient way to declare as many variables as you want using arrays. Arrays are defined in number of ways:

1. An array is a group of related data items that share a common name.
2. An array is a collection of elements of same type which are stored in continuous locations in memory. For example, suppose we want to store marks of 50 students or salaries of 25 employees, we can use arrays. In short, a list of data items of same type can be stored in an array. Arrays can be single or multidimensional. Let us first look into single dimensional arrays.

2.1.1 Defining an Array

We can declare any array variable to store them. Array is declared as

```
n = int(input("Enter the number of elements: "))
a = []
for i in range(n):
    a.append(int(input(f"Enter element {i + 1}: ")))
```

It means we declared 100 variables of integer type. These variables have the same name a. The size can be any integer constant. The above declaration reserves 100 locations in the memory to store integer numbers.



Fig. 2.2: Memory allocation for single dimensional array

These locations are a[0], a[1], ..., a[99] are called elements of array. They are just like any other variable names.

2.1.2 Reading into an Array

To read elements into an array we can use for loop as,

```
    a = [0] * n
for i in range(n):
    a[i] = int(input())
```

where n is the number of elements in the array, n can be any number less than size of the array. When i=0, first element entered from keyboard gets stored in the location a[0], when i=1 second number entered is stored in location a[1] and so on.

Similarly, if array elements are real (say float x[100]) then we can write,

```
    a = [0] * n
for i in range(n):
    a[i] = int(input())
```

2.1.3 Printing an Array

The elements in the array can be displayed using for loop as

```
for value in a:
        print(value)
```

Thus, when i = 0 the element stored in the location a[0] gets displayed. When i = 1, the second element in the array gets displayed and so on.

2.1.4 Processing an Array

Array elements can be processed the same way we process other variables i.e. just like we write c = a + b; we can write,

```
a = [0] * 11  # make sure the list is large enough for a[10]
```

Numbers stored at location a[0] and a[1] in the array are added and the sum is stored in the location a[2].

Similarly we can have,

```
a[0] = 5
a[1] = 7
a[2] = a[0] + a[1]
sum_value = a[0] + a[1]
avg = (a[0] + a[1]) / 2
a[10] = 20
a[5] = 4
a[9] = a[8]  # assuming a[8] is already assigned.
```

It means we can take any element in an array randomly and process it.

2.2 EXAMPLES ON ARRAY

Now let us write some programs to understand how single dimensional arrays are used.

Program 2.1: To Store and Display the Elements in an Array.

```
a = [0] * 10
# Store elements
for i in range(10):
   a[i] = i * 10
# Display elements
for i in range(10):
   print(a[i])
```

Explanation:

1. The declaration int a[10]; will reserve 10 integer locations a[0], a[1], a[2], ... a[9].
2. The first for loop puts values 0, 10, 20, ..., 90 in the locations a[0], a[1], a[2], ...a[9].
3. The second for loop displays the values 0, 10, 20, ..., 90.

Program 2.2: To Display First 10 Fibonacci Numbers.

```
a = [0] * 10

a[0] = 0
a[1] = 1

for i in range(2, 10):
   a[i] = a[i - 1] + a[i - 2]

for i in range(10):
   print(a[i])
```

Explanation:

1. The declaration int a[10]; will reserve 10 integer locations a[0], a[1], a[2], ... a[9].
2. Since first two Fibonacci numbers are 0 and 1 they are stored in a[0] and a[1].
3. The first for loop starts with i=2 and calculates the Fibonacci number.
4. The second for loop displays all the Fibonacci numbers.
5. The output will be 0 1 1 2 3 5 8 13 21 34.

Program 2.3: To Accept Numbers from Users and Display them in Reverse Order.

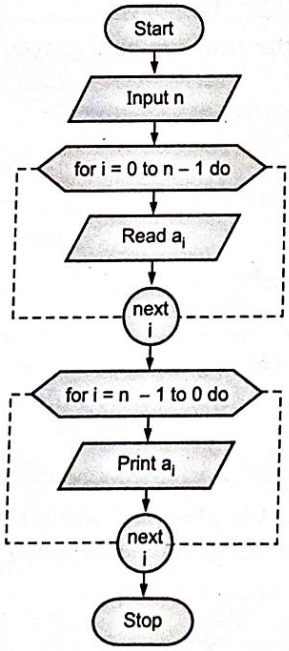


Fig. 2.3: Flow chart for program 2.3

```
n = int(input("Enter how many numbers you have? "))

a = [0] * n
print("Enter the numbers:")
for i in range(n):
   a[i] = int(input())

print("The reverse list is:")
for i in range(n - 1, -1, -1):
   print(a[i])
```

Explanation:

1. First, for loop accepts numbers from users and stores them into an array.
2. Since, the last numbers will be in an $(n-1)^{th}$ location, the second for loop which displays the elements starts with i = n–1 and goes upto 0.

Note: Most of the array related programs will have 3 parts, viz. reading the elements in an array, processing the array and printing results.

Program 2.4: To Count Number of Odd and Even Numbers in a List.

```
n = int(input("Enter how many numbers? "))
a = [0] * n
print("Enter the numbers:")
for i in range(n):
   a[i] = int(input())
ec = 0  # even count
oc = 0  # odd count
for i in range(n):
   if a[i] % 2 == 0:
      ec += 1
   else:
      oc += 1
print(f"Even numbers are {ec}")
print(f"Odd numbers are {oc}")
```

Program 2.5: To Count Number of Positive Numbers, Negative Numbers and Zeros in a List.

```
n = int(input("Enter how many numbers? "))

a = [0] * n
print("Enter the numbers:")
for i in range(n):
    a[i] = int(input())

pc = 0  # positive count
nc = 0  # negative count
zc = 0  # zero count

for i in range(n):
    if a[i] > 0:
        pc += 1
    elif a[i] < 0:
        nc += 1
    else:
        zc += 1

print(f"Number of positive numbers = {pc}")
print(f"Number of negative numbers = {nc}")
print(f"Number of zeros = {zc}")
```

Program 2.6: To Find Sum, Average, and Standard Deviation of n Numbers.

Explanation: We can write a program to find sum and average of numbers without using array. But to find standard deviation, we have to have an array because the numbers accepted from the user will be required second time to find standard deviation. It is given as

$$sd = \sqrt{\frac{\sum_i (a_i - \bar{a})^2}{n}}$$

where, $\bar{a}$ is Average of the umbers.

Hence to find standard deviation, we have to first find average and then find the $\sum_i (a_i - \bar{a})^2$ where, i = 0 to n – 1.

```
import math
n = int(input("Enter how many numbers? "))
a = [0.0] * n
print("Enter the numbers:")
for i in range(n):
    a[i] = float(input())

# Calculate sum and average
s = sum(a)
avg = s / n

print(f"Sum is {s}")
print(f"Avg is {avg}")

# Calculate standard deviation
squared_diff_sum = 0
for i in range(n):
    squared_diff_sum += (a[i] - avg) ** 2

sd = math.sqrt(squared_diff_sum / n)
print(f"Standard deviation is {sd}")
```

Program 2.7: To Find Maximum of given n Numbers.

```
n = int(input("Enter how many numbers? "))

a = [0] * n
print("Enter the numbers:")
for i in range(n):
    a[i] = int(input())

max_val = a[0]
for i in range(1, n):
    if a[i] > max_val:
        max_val = a[i]

print(f"Maximum is {max_val}")
```

Program 2.8: To Search a Number from a given List of Numbers.

```
x = int(input("Enter the number: "))

a = []  # to store binary digits manually
q = x
while q > 0:
    a.append(q % 2)
    q = q // 2

print("The Binary number is ....")
for bit in reversed(a):
    print(bit, end='')

print()  # newline after output
```

Program 2.9: To Convert Decimal Number into Binary.

```
void main( )
{
    int a[50];
    int i, x, q;
    printf("Enter the number \n");
        scanf("%d", &x);
    i=0; q=x;
    while (q>0)
    {
        a[i] = q%2;
        q = q/2;
        i++;
    }
    n=i;
    printf("The Binary number is .... \n");
    for(i=n-1;i>=0;i--)
        printf("%d", a[i]);
    getch( );
}
```

2.2.1 Initializing an Array

Just like we initialize memory variables at the time of declaration e.g., int x = 10; array elements can also be initialized as

```
a = [10, 20, 30, 40]
```

It will initialize the elements of the array a as

```
a = [0] * 4  # initialize list with 4 elements
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
```

Here note that there is no need to declare size, since we are specifying the data to be stored in array. Let us consider following example.

Program 2.10: Initializing an Array.

```
a = [10, 20, 30, 40]
for i in range(4):
  a[i] += 10
for i in range(4):
  print(a[i])
```

Output :

```
20
30
40
50
```

2.3 ARRAY AS AN ABSTRACT DATA TYPE

Abstract data types, commonly abbreviated ADTs, are a way of classifying data structures based on how they are used and the behaviors they provide. They do not specify how the data structure must be implemented but simply provide a minimal expected interface and set of behaviors.

Data structure is a concrete implementation of a data type. It's possible to analyze the time and memory complexity of a Data Structure but not from a data type. The Data Structure can be implemented in several ways and its implementation may vary from language to language.

Array - Abstract Data Type

The array is an Abstract Data Type (ADT) that holds a collection of elements accessible by an index.

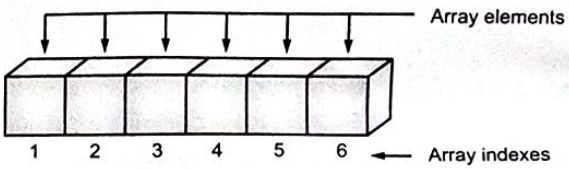


Fig. 2.4: One-dimensional array with six elements

The elements stored in an array can be anything from primitives types such as integers to more complex types like instances of classes.

An element is stored in a given index and they can be retrieved at a later time by specifying the same index.

The array (ADT) has one property, they store and retrieve elements using an index. The array (ADT) is usually implemented by an Array (Data Structure).

The way indices work is specific to the implementation, but you can usually think of them as the slot number in the array that the value occupies.

ADT - Minimal Required Functionality

To fulfill the ADT an array must implement at least a way to retrieve the data from a given index and to store the data in a given index.

```
set(i, v) -> Sets the value of index i to v
get(i) -> Returns the value of index i in the array V
```

Since abstract data types don't specify an implementation, this means it's also incorrect to talk about the time complexity of a given abstract data type.

ADT — Operations

Traversal : You must be thinking, "how could I print all the elements of an array?" An array can be `traversed`, is possible to get the elements in the array by the index we only need to know what is the size of the array:

```
array = Array("integers", (1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
index = 1   # In most implementations, arrays starts from 0
while the index is less than size(array)
  print array.get(index)
  index + 1
```

This is an abstract operation and the implementation will define how you access the array and iterate over them.

Search: You must be thinking, "how do I find an element inside the array?"

Is possible to `traverse` the array and compare if the element we are looking for exists in the array:

```
array = Array("integers", (1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
targetValue = 12 index = 1    # In most implementations, arrays
starts from 0 while index is less than size(array)
  value = array.get(index)  if value is equal to targetValue
    print Element found, index   index + 1
```

This is an example of how we could search for an element inside an array, but this is not the optimal way. There are different and, in some cases, better algorithms to `traverse` and `search` in arrays.

Implementation

The simplest implementation of an array only stores the memory address of its first index. Some languages also store the size of the array to prevent buffer overflow errors. Insertion and retrieval of an item at a given index is performed by taking the index I, multiplying by the item size S in the number of bytes, and adding it to the base memory address B.

For zero-based numbering languages, the memory address M of an index is given by:

```
M = B + SI
```

Arrays can be implemented in different ways, let's take a look at how arrays are implemented in C:

Arrays in C are implemented to hold a single type of data. This restriction exists due the way arrays are stored in memory. To create an array in C, we must declare its `type` and `size`, this information will be used to compute the needed memory space and to allocate the array accordingly.

```
int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

2.4 OPERATIONS ON ARRAY

Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.

- **Element:** Each item stored in an array is called an element.
- **Index:** Each location of an element in an array has a numerical index, which is used to identify the element.

Array Representation

Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.

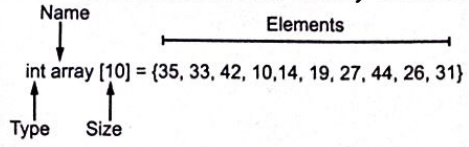


Fig. 2.5

Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.

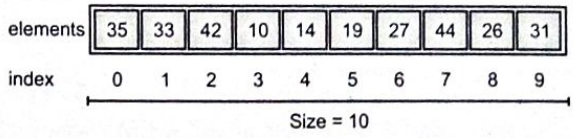


Fig. 2.6

As per the above illustration, following are the important points to be considered.

- Index starts with 0.
- Array length is 10 which means it can store 10 elements.
- Each element can be accessed via its index. For example, we can fetch an element at index 6 as 9.

Basic Operations

Following are the basic operations supported by an array.

- **Traverse:** Print all the array elements one by one.
- **Insertion:** Adds an element at the given index.
- **Deletion:** Deletes an element at the given index.
- **Search:** Searches an element using the given index or by the value.
- **Update:** Updates an element at the given index.

In C, when an array is initialized with size, then it assigns defaults values to its elements in following order:

Data Type	Default Value
bool	False
char	0
Int	0
Float	0.0
Double	0.0f
Void	
wchar_t	0

Traverse Operation

This operation is to traverse through the elements of an array.

Program 2.11: Program Traverses and Print the Elements of an Array.

```
LA = [1, 3, 5, 7, 8]
item = 10
k = 3
n = 5
print("The original array elements are:")
for i in range(n):
  print(f"LA[{i}] = {LA[i]}")
```

When we compile and execute the above program, it produces the following result:

Output:

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

Insertion Operation

Insert operation is to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array.

Here, we see a practical implementation of insertion operation, where we add data at the end of the array.

Program 2.12: Program Insert Array and Print the Elements of an Array.

Following is the implementation of the above algorithm:

```
LA = [1, 3, 5, 7, 8]
item = 10
k = 3
n = 5

print("The original array elements are:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")

LA.append(0)  # Extend list by one to make space
j = n
while j >= k:
   LA[j] = LA[j - 1]
   j -= 1

LA[k] = item
n += 1

print("The array elements after insertion:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")
```

When we compile and execute the above program, it produces the following result:

Output :

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

The array elements after insertion :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 10

LA[4] = 7

LA[5] = 8

Deletion Operation

Deletion refers to removing an existing element from the array and re-organizing all elements of an array.

Algorithm 2.1

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to delete an element available at the Kth position of LA.

1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set LA[J] = LA[J + 1]
5. Set J = J+1
6. Set N = N-1
7. Stop

Program 2.13: Program to Delete Array and Print the Elements of an Array.

```
LA = [1, 3, 5, 7, 8]
k = 3
n = 5

print("The original array elements are:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")
j = k
while j < n:
   LA[j - 1] = LA[j]
   j += 1
n -= 1
LA = LA[:n]  # trim the list to new size
print("The array elements after deletion:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")
```

When we compile and execute the above program, it produces the following result:

Output :

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

The array elements after deletion :

LA[0] = 1

LA[1] = 3

LA[2] = 7

LA[3] = 8

Search Operation

You can perform a search for an array element based on its value or its index.

Algorithm 2.2

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to find an element with a value of ITEM using sequential search.

1. Start
2. Set J = 0
3. Repeat steps 4 and 5 while J < N
4. IF LA[J] is equal ITEM THEN GOTO STEP 6
5. Set J = J +1
6. PRINT J, ITEM
7. Stop

Program 2.14: Program to Search Array and Print the Elements of an Array.

```
LA = [1, 3, 5, 7, 8]
item = 5
n = 5
j = 0

print("The original array elements are:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")

while j < n:
   if LA[j] == item:
      break
   j += 1

print(f"Found element {item} at position {j + 1}")
```

When we compile and execute the above program, it produces the following result –

Output :

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

Found element 5 at position 3

Update Operation

Update operation refers to updating an existing element from the array at a given index.

Algorithm 2.3

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to update an element available at the Kth position of LA.

1. Start
2. Set LA[K-1] = ITEM
3. Stop

Program 2.15: Program to Update Array and Print the Elements of an Array.

```
LA = [1, 3, 5, 7, 8]
k = 3
n = 5
item = 10

print("The original array elements are:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")

LA[k - 1] = item

print("The array elements after updation:")
for i in range(n):
   print(f"LA[{i}] = {LA[i]}")
```

When we compile and execute the above program, it produces the following result –

Output :

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

The array elements after updation :

LA[0] = 1

LA[1] = 3

LA[2] = 10

LA[3] = 7

LA[4] = 8

2.5 STORAGE REPRESENTATION AND THEIR ADDRESS CALCULATION: ROW MAJOR AND COLUMN MAJOR

2.5.1 Address Calculation in Single (One) Dimension Array

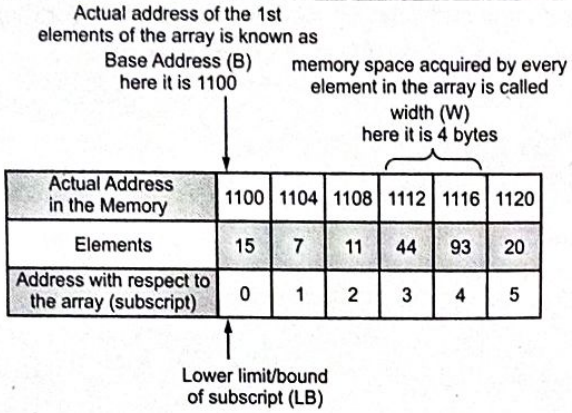


Fig. 2.7

Array of an element of an array say "A[I]" is calculated using the following formula:

Address of A [I] = B + W * (I – LB)

Where,

B = Base address

W = Storage Size of one element stored in the array (in byte)

I = Subscript of element whose address is to be found

LB = Lower limit / Lower Bound of subscript, if not specified assume 0 (zero)

SOLVED EXAMPLES

Example 2.1: *Given the base address of an array B[1300.....1900] as 1020 and size of each element is 2 bytes in the memory. Find the address of B[1700].*

Solution:

The given values are: B = 1020, LB = 1300, W = 2, I = 1700

Address of A [I] = B + W * (I – LB)

= 1020 + 2 * (1700 – 1300)

= 1020 + 2 * 400

= 1020 + 800

= 1820

2.5.2 Address Calculation in Double (Two) Dimensional Array

While storing the elements of a 2-D array in memory, these are allocated contiguous memory locations. Therefore, a 2-D array must be linearized so as to enable their storage. There are two alternatives to achieve linearization: Row-Major and Column-Major.

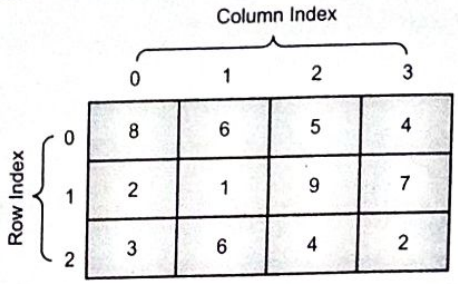
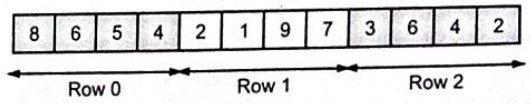


Fig. 2.8 : Two-Dimentional array

Row-Major [Row Wise Arrangement]



Column-Major [Column Wise Arrangement]

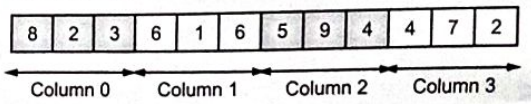


Fig. 2.9

Address of an element of any array say "A[I][J]" is calculated in two forms as given:

1. Row Major System
2. Column Major System

1. Row Major System:

The address of a location in Row Major System is calculated using the following formula:

Address of A [I][J] = B + W * [N *(I – Lr) +(J – Lc)]

2. Column Major System:

The address of a location in Column Major System is calculated using the following formula:

Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]

Where,

B = Base address

I = Row subscript of element whose address is to be found

J = Column subscript of element whose address is to be found

W = Storage Size of one element stored in the array (in byte)

Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)

Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)

M = Number of rows of the given matrix

N = Number of columns of the given matrix

Important : Usually, number of rows and columns of a matrix are given (like A[20][30] or A[40][60]) but if it is given as A[Lr- - - - - Ur, Lc- - - - - Uc].

In this case number of rows and columns are calculated using the following methods:

Number of rows (M) will be calculated as = (Ur – Lr) + 1

Number of columns (N) will be calculated as = (Uc – Lc)+ 1

And rest of the process will remain same as per requirement (Row Major Wise or Column Major Wise).

Example 2.2: *An array X [-15.....10, 15.....40] requires one byte of storage. If beginning location is 1500 determine the location of X [15][20].*

Solution:

As you see here the number of rows and columns are not given in the question. So, they are calculated as:

Number or rows say M = (Ur – Lr) + 1

= [10 – (- 15)] +1 = 26

Number or columns say N = (Uc – Lc) + 1

= [40 – 15)] +1 = 26

1. Column Major Wise Calculation of above Equation

The given values are: B = 1500, W = 1 byte, I = 15, J = 20, Lr = -15, Lc = 15, M = 26

Address of A [I][J] = B + W * [(I – Lr) + M * (J – Lc)]

= 1500 + 1 * [(15 – (-15)) + 26 * (20 – 15)]

= 1500 + 1 * [30 + 26 * 5]

= 1500 + 1 * [160] = 1660

2. Row Major Wise Calculation of above Equation

The given values are: B = 1500, W = 1 byte, I = 15, J = 20, Lr = -15, Lc = 15, N = 26

Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]

= 1500 + 1* [26 * (15 – (-15))) + (20 – 15)]

= 1500 + 1 * [26 * 30 + 5] = 1500 + 1 * [780 + 5] = 1500 + 785

= 2285

2.6 MULTIDIMENSIONAL ARRAYS: TWO-DIMENSIONAL ARRAYS

Sometimes, we have a group of numbers represented in terms of rows and columns. e.g., table of values.

Table 2.1

	Item 1	Item 2	Item 3
S_1	10	40	10
S_2	20	30	20
S_3	30	50	50
S_4	10	20	60

In mathematics, this table of values is represented in terms of matrix. It consists of rows and columns. The table given above can be represented in C using two dimensional array as

```
table = [[0 for _ in range(3)] for _ in range(4)]
```

Thus, two dimensional array can be declared as

```
a = [[0 for _ in range(10)] for _ in range(10)]
```

It will reserve 100 locations in the memory to store 100 integer numbers. The names of the locations will be

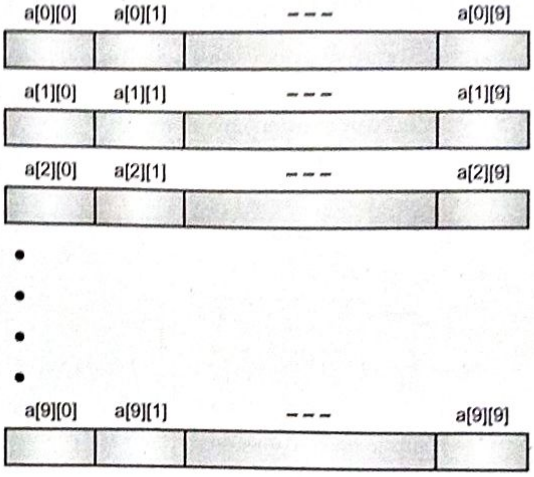


Fig. 2.10: Memory allocation for 2-D array

2.6.1 Reading into Two Dimension Array

To read data in single dimensional array, we use single for loop. Here, we have to use nested for loop. For example, if an array consists of m rows and n columns. We can write

```
a = [[0] * n for _ in range(m)]
for i in range(m):
    for j in range(n):
        a[i][j] = int(input())
```

The outer for loop runs for m times and the inner n times. The elements will be accepted row wise. This is called as row major representation of the two dimensional array.

2.6.2 Displaying the Two Dimensional Array

To print the elements stored in two dimensional arrays, nested or loop will be required as follows:

```
for i in range(m):
    for j in range(n):
        print(a[i][j], end=' ')
    print()
```

2.6.3 Processing Two Dimensional Arrays

Just like we access any element in a single dimensional array randomly, we can access any element in two dimensional array for storage of value or calculation purpose.

e.g. a[0][0] = 10;

sum = a[0][0] + a[1][1] + a[2][2];

a[0][0] = a[0][0] + b[0][0];

a[2][2] = a[0][0] + a[1][1];

2.7 EXAMPLES ON TWO DIMENSIONAL ARRAYS

Now let us write some programs based on two dimensional arrays.

Program 2.16: To Find Sum of Two Matrices.

There will be three matrices involved here. Three two dimensional arrays will be required to store the three matrices. The addition is carried out element by element.

i.e., c[i][j] = a[i][j] + b[i][j];

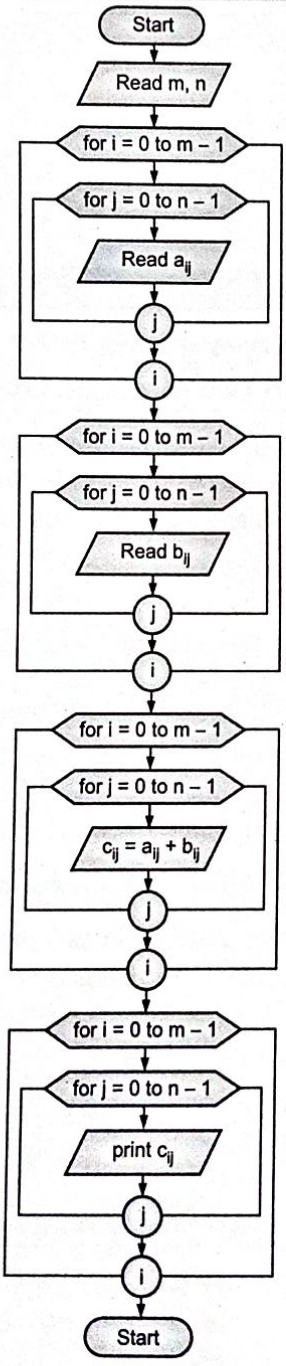


Fig. 2.11: Flow chart for matrix addition

```
m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))
a = [[0] * n for _ in range(m)]
b = [[0] * n for _ in range(m)]
c = [[0] * n for _ in range(m)]
print("Enter first matrix:")
for i in range(m):
    for j in range(n):
        a[i][j] = int(input())
print("Enter second matrix:")
for i in range(m):
    for j in range(n):
        b[i][j] = int(input())
# Add the matrices
for i in range(m):
    for j in range(n):
        c[i][j] = a[i][j] + b[i][j]
print("Resultant matrix is:")
for i in range(m):
    for j in range(n):
        print(c[i][j], end=' ')
    print()
```

Explanation:

1. Order means number of rows and columns of the two matrices to be added. It is same for both matrices hence same variables m and n are used.
2. The two matrices are accepted from user and stored in 2-D arrays a and b.
3. The third nested for loop adds the elements of a and b and stores the result in c.
4. The matrix c is displayed by the 4"h Nested for.

Program 2.17: To Find Sum of all Major and Minor Diagonal Elements in a given Matrix.

```
m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

a = [[0] * n for _ in range(m)]

print("Enter the matrix:")
for i in range(m):
    for j in range(n):
        a[i][j] = int(input())
s = 0  # Major diagonal sum
s1 = 0  # Minor diagonal sum
for i in range(m):
    for j in range(n):
        if i == j:
            s += a[i][j]
        if i + j == m - 1:
            s1 += a[i][j]
print(f"Sum of major diagonal: {s}, minor diagonal: {s1}")
```

Explanation:

Diagonal elements will have row and column number same hence the condition i == j is tested for adding elements.

Program 2.18: Write a Program to Find Transpose of a Matrix.

```
m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

a = [[0] * n for _ in range(m)]
b = [[0] * m for _ in range(n)] # Transpose will be n x m

print("Enter the matrix:")
for i in range(m):
    for j in range(n):
        a[i][j] = int(input())

# Transpose logic
for i in range(m):
    for j in range(n):
        b[j][i] = a[i][j]

print("The transpose is:")
for i in range(n):
    for j in range(m):
        print(b[i][j], end=' ')
    print()
```

Explanation:

1. Array a store the matrix of the order m x n whose transpose is to be stored in matrix b.
2. After reading the elements in a, the elements of a are copied to b one by one. The element in the ith row and j^{th} column of a is copied to j^{th} row and i^{th} column of b.
3. Note that third loop which prints the elements of b has n first and m next because the transpose will have order $n \times m$. i.e., it will haven rows and m columns. e.g. Transpose of 3×2 will be 2×3 matrix.
4. We cannot write the statement b[i][j] = a[j][i] instead of b[j][i] = a[i][j]. Find out the reason.

Program 2.19: To Print Product of Two Matrices.

```
m = int(input("Enter number of rows for 1st matrix: "))
n = int(input("Enter number of columns for 1st matrix: "))

p = int(input("Enter number of rows for 2nd matrix: "))
l = int(input("Enter number of columns for 2nd matrix: "))

if n != p:
    print("Invalid order: Columns of 1st matrix must equal rows of 2nd matrix.")
    exit()
a = [[0] * n for _ in range(m)]
b = [[0] * l for _ in range(p)]
c = [[0] * l for _ in range(m)]

print("Enter 1st matrix:")
for i in range(m):
    for j in range(n):
        a[i][j] = int(input())

print("Enter 2nd matrix:")
for i in range(p):
    for j in range(l):
        b[i][j] = int(input())

# Matrix multiplication
for i in range(m):
    for j in range(l):
        c[i][j] = 0
        for k in range(n):
            c[i][j] += a[i][k] * b[k][j]

print("Product is:")
for i in range(m):
    for j in range(l):
        print(c[i][j], end=' ')
    print()
```

Explanation:

1. First matrix is stored in array a and has order $m \times n$. second matrix is stored in array b and has order $p \times l$.
2. Number of columns of first matrix should be equal to number of rows of second matrix. Hence, if n!=p the error message "Invalid matrices" is displayed and exit(0) statement terminates the program.
3. The process of multiplication will require Nested for loops. The outer for loop with variable 1st will keep on changing the row number of 2nd matrix. The second for loop with variable j will keep on changing column of 2nd matrix. The third for loop will do the sum of multiplication of ith row elements of first matrix and j^{th} column element of second matrix and the result will be stored in 3rd matrix's i^{th} row and j^{th} column.

   e.g., when i = 0, j = 0, k runs from 0 to n − 1. The value a[0][0] * b[0][0] + a[0][1] * b[1][0] + ... + a[0][n − 1] * b[n − 1][0] will be stored in the location c[0][0] and so on.
4. The resultant matrix c is of the order $m \times l$. Hence, the last nested for loops run for $m \times l$ times.

2.7.1 Initializing Two Dimensional Array

Just like we initialize one dimensional array, we can initialize two dimensional arrays as follows:

```
a = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]
```

It means we are storing the numbers in the array as follows:

a[0][0] =10 a[1][0] =40
a[0][1] = 20 a[2][1] =50
a[0][2] =30 a[1][2] =60 etc.

Note that it is necessary to have column size mentioned as a[][3] otherwise it will become difficult for the compiler to allocate memory.

If number of columns is not specified, the array can be initialized as

```
a = [
[10, 20, 30],
[40, 50, 60],
[70, 80, 90]
]
```

Program 2.20: To Find Sum of Elements in a Matrix.

```
a = [
  [10, 20, 30],
  [40, 50, 60],
  [70, 80, 90]
]

s = 0
for i in range(3):
  for j in range(3):
    s += a[i][j]

print(f"Sum is {s}")
```

2.8 N - DIMENSIONAL ARRAY [Oct. 22]

We can have more than two dimensional arrays as well. For example, a three dimensional array can be declared as

```
int a[2][2][3];
```

There will be total 12 elements in this array. These elements will be a[0][0][0], a[0][0][1], a[0][0][2], a[0][1][1], a[0][1][2], a[1][0][0], a[1][0][1], a[1][0][2], a[1][1][0], a[1][1][1], a[1][1][2].

While initializing the multidimensional array the leftmost dimensional can be omitted.

e.g. int a[ ][2][3] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}

2.9 SPARSE MATRIX [Oct. 22]

A matrix is a two-dimensional data object made of m rows and n columns, therefore having total m x n values. If most of the elements of the matrix have 0 value, then it is called a sparse matrix.

- **Storage:** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing Time:** Computing time can be saved by logically designing a data structure traversing only non-zero elements.

Example:

0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0

Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases. So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with Triples: (Row, Column, Value).

2D array is used to represent a sparse matrix in which there are three rows named as

1. **Row:** Index of row, where non-zero element located
2. **Column:** Index of column, where non-zero element located
3. **Value:** Value of the non-zero element located, index – (row, column)

$$\begin{bmatrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{bmatrix} \longrightarrow$$

Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Fig. 2.12

C Program to check matrix is a sparse matrix example

This program for the sparse matrix in c allows the user to enter the number of rows and columns of a Matrix. Next, we are going to check whether the given matrix is sparse or not using For Loop.

Program 2.21: TO Check whether the given Matrix Sparse Matrix or Not.

```
rows = int(input("Enter number of rows: "))
columns = int(input("Enter number of columns: "))
a = [[0] * columns for _ in range(rows)]

print("Enter the matrix elements:")
for i in range(rows):
  for j in range(columns):
    a[i][j] = int(input())
total = 0
for i in range(rows):
  for j in range(columns):
    if a[i][j] == 0:
      total += 1
if total > (rows * columns) // 2:
  print("The matrix you entered is a Sparse Matrix")
else:
  print("The matrix you entered is NOT a Sparse Matrix")
```

Analysis :

In this sparse matrix in python Program, we declare single two dimensional arrays Multiplication of size of 10 * 1

2.10 SPARSE MATRIX PROGRAM [Oct. 22]

```
MAX = 20

def read_sparse():
    b = []
    m = int(input("Enter number of rows: "))
    n = int(input("Enter number of columns: "))
    t = int(input("Enter number of non-zero elements: "))
    b.append([m, n, t])
    for _ in range(t):
        row, col, val = map(int, input("Enter the triple (row column value): ").split())
        b.append([row, col, val])
    return b

def add_sparse(b1, b2):
    if b1[0][0] != b2[0][0] or b1[0][1] != b2[0][1]:
        print("Invalid matrix sizes! Must be same dimensions.")
        exit()

    t1 = b1[0][2]
    t2 = b2[0][2]
    i = j = 1
    k = 1

    b3 = [[b1[0][0], b1[0][1], 0]]  # header for result

    while i <= t1 and j <= t2:
        if b1[i][0] < b2[j][0] or (b1[i][0] == b2[j][0] and b1[i][1] < b2[j][1]):
            b3.append(b1[i])
            i += 1
        elif b2[j][0] < b1[i][0] or (b2[j][0] == b1[i][0] and b2[j][1] < b1[i][1]):
            b3.append(b2[j])
            j += 1
        else:
            # same row and column
            summed = b1[i][2] + b2[j][2]
            if summed != 0:
                b3.append([b1[i][0], b1[i][1], summed])
            i += 1
            j += 1

    # remaining elements
    while i <= t1:
        b3.append(b1[i])
        i += 1
    while j <= t2:
        b3.append(b2[j])
        j += 1

    b3[0][2] = len(b3) - 1
    return b3

def print_sparse(b):
    print("\nRow\tCol\tValue")
    for i in range(1, b[0][2] + 1):
        print(f"{b[i][0]}\t{b[i][1]}\t{b[i][2]}")

# Main Program
print("Enter first sparse matrix:")
b1 = read_sparse()

print("\nEnter second sparse matrix:")
b2 = read_sparse()

b3 = add_sparse(b1, b2)

print("\nResultant sparse matrix after addition:")
print_sparse(b3)
```

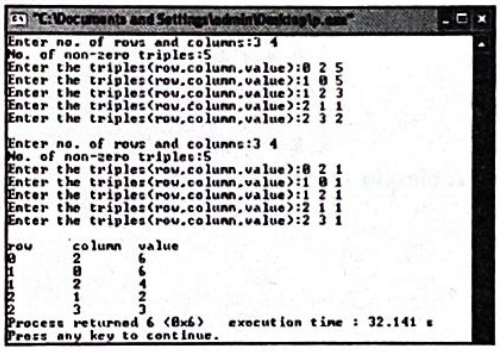


Fig. 2.13

2.11 TRANSPOSE OF A SPARSE MATRIX

[April 23]

To transpose a matrix, we can simply change every column value to the row value and vice-versa, however, in this case, the resultant matrix won't be sorted as we require. Hence, we initially determine the number of elements less than the current element's column being inserted in order to get the exact index of the resultant matrix where the current element should be placed. This is done by maintaining an array index[] whose i^{th} value indicates the number of elements in the matrix less than the column i.

2.11.1 Simple Transpose

Simple transpose method is rather simple to learn and understand (as the name suggests). Here Algorithm goes through each row of the sparse matrix by looking through all terms for once in the column. Its Time complexity is O(Number of columns * Number of values).

Suppose we have a sparse matrix

$$\begin{bmatrix} 3 & 3 & 6 \\ 0 & 0 & 2 \\ 0 & 1 & 6 \\ 1 & 0 & 1 \\ 1 & 2 & 9 \\ 2 & 1 & 4 \\ 2 & 2 & 5 \end{bmatrix}$$

We require three variables say (I,j, k), so *i* takes values from 0-3 here (number of columns ([0][1])), j takes values from 1-6 (max number of elements ([0][2]) and it starts from 1 because we put values in sparse matrix from second row) while k starts from 1.

So, what happens is it first go, it will check if 0 is present in the column of sparse matrix. If yes, it will swap values in resultant matrix. So, in first go, matrix will be like

$$\begin{bmatrix} 3 & 3 & 2 \\ 0 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}$$

In second go, it will check for I = 1 (column for value 1 in sparse) and after swapping resultant matrix will be like

$$\begin{bmatrix} 3 & 3 & 4 \\ 0 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 0 & 6 \\ 1 & 2 & 4 \end{bmatrix}$$

And after checking for I = 2 (column for value 2 in sparse), the resultant matrix will be

$$\begin{bmatrix} 3 & 3 & 6 \\ 0 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 0 & 6 \\ 1 & 2 & 4 \\ 2 & 1 & 9 \\ 2 & 2 & 5 \end{bmatrix}$$

Program 2.22: Simple Transpose Matrix.

```
def accept():
    global r, c
    r = int(input("Enter number of rows: "))
    c = int(input("Enter number of columns: "))
    a = [[0 for _ in range(c)] for _ in range(r)]
    print("Enter the elements:")
    for i in range(r):
        for j in range(c):
            a[i][j] = int(input())
    return a

def display(matrix, rows, cols):
    for i in range(rows):
        for j in range(cols):
            print(matrix[i][j], end="\t")
        print()

def convert_to_sparse(a):
    b = []
    count = 0
    for i in range(len(a)):
        for j in range(len(a[0])):
            if a[i][j] != 0:
                b.append([i, j, a[i][j]])
                count += 1
    # First row contains metadata: [rows, cols, non-zero count]
    b.insert(0, [len(a), len(a[0]), count])
    return b

def simple_transpose(sparse):
    rows, cols, non_zero = sparse[0]
    transpose = [[cols, rows, non_zero]] # Metadata

    if non_zero > 0:
        for i in range(cols):  # Transpose by column
            for j in range(1, non_zero + 1):
                if sparse[j][1] == i:
                    transpose.append([sparse[j][1], sparse[j][0],
sparse[j][2]])
    return transpose

# Main execution
a = accept()

print("\nThe conventional matrix is:")
display(a, r, c)

b = convert_to_sparse(a)
print("\nThe sparse matrix is:")
display(b, len(b), 3)

c = simple_transpose(b)
print("\nThe transpose matrix (sparse):")
display(c, len(c), 3)
```

2.11.2 Fast Transpose

As its name suggests, it is a faster way to transpo sparse and also a little bit hard to understand. T complexity is O(Number of columns + Number of ter Here, we require 2 arrays, namely, count and position.

Count array mainly stores the values of columns pres in sparse matrix. E.g. If we have conventional matrix of [3 x 4], then our count array will be {0,1,2,3,4}

Position array stores the position of the first occurrenc the values present in column of sparse. It is later incremen in the program.

Program 2.23: Fast Transpose Matrix.

```
MAX = 100

class Sparse:
    def __init__(self, row=0, col=0, val=0):
        self.row = row
        self.col = col
        self.val = val

def readsparse(m, n):
    a = [Sparse() for _ in range(MAX)]
    a[0].row = m
    a[0].col = n
    k = 1

    print("Enter the elements:")
    for i in range(m):
        for j in range(n):
            item = int(input())
            if item == 0:
                continue
            a[k].row = i
            a[k].col = j
            a[k].val = item
            k += 1

    a[0].val = k - 1

    print("\nThe entered sparse matrix is:")
    print("Row\tColumn\tValue")
    for p in range(k):
        print(f"{a[p].row}\t{a[p].col}\t{a[p].val}")

    return a

def fast_transpose(a):
    b = [Sparse() for _ in range(MAX)]
    row_terms = [0] * MAX
    start_pos = [0] * MAX

    num_terms = a[0].val
    num_cols = a[0].col

    b[0].row = num_cols
    b[0].col = a[0].row
    b[0].val = num_terms

    if num_terms > 0:
        for i in range(num_cols):
            row_terms[i] = 0

        for i in range(1, num_terms + 1):
            row_terms[a[i].col] += 1

        start_pos[0] = 1
        for i in range(1, num_cols):
            start_pos[i] = start_pos[i - 1] + row_terms[i - 1]

        for i in range(1, num_terms + 1):
            j = start_pos[a[i].col]
            start_pos[a[i].col] += 1
            b[j].row = a[i].col
            b[j].col = a[i].row
            b[j].val = a[i].val

    print("\nThe Fast Transpose sparse matrix is:")
    print("Row\tColumn\tValue")
    for p in range(num_terms + 1):
        print(f"{b[p].row}\t{b[p].col}\t{b[p].val}")

# Main Execution
m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

a = readsparse(m, n)
fast_transpose(a)
```

Output:

Enter the no of rows and columns: 6 6

Enter the elements:

15	0	0	22	0	-15
0	11	3	0	0	0
0	0	0	-6	0	0
0	0	0	0	0	0
91	0	0	0	0	0
0	0	28	0	0	0

The entered sparse matrix is:

Row	Column	Value
6	6	8
0	0	15
0	3	22
0	5	-15
1	1	11
1	2	3
2	4	-6
4	1	91
5	3	28

The Fast Transpose sparse matrix is:

Row	Column	Value
6	6	8
0	0	15
1	1	11
1	4	91
2	1	3
3	0	22
3	5	28
4	2	-6
5	0	-15

2.12 SEARCHING TECHNIQUES

Sorting and Searching are fundamental operations in computer science. Sorting refers to the operation of arranging data in some given order. Searching refers to the operation of searching the particular record from the existing information.

Consider a database of banking system where information of all customers such as name, contact number, address and account number is stored. If a manager wants to search for a record of a particular customer, he has to look for that record from among all records that has been stored in a database. This process of looking up for a particular record in a database is referred as searching.

If records in a banking database are not ordered properly it will be very difficult for a manager to search for a specific record. On the contrary if all the records are arranged in order according to some criteria like names in alphabetical order or account numbers in ascending order, searching becomes easy and fast. The process of ordering the records in a data base is called sorting. Thus, efficient searching sorting is necessary.

Searching Techniques are basically categorized into:

1. **Sequential Search:** A method for finding a target value within a list.
2. **Sentinel Search:** Sentinel Linear Search as the name suggests is a type of Linear Search where the numbers of comparisons are reduced as compared to a traditional linear search.
3. **Binary Search:** This is a search algorithm that finds the position of a target value within a sorted array. It compares the target value to the middle element of the array; if they are unequal, the half in which the target cannot lie is eliminated and the search continues on the remaining half until it is successful.
4. **Fibonacci Search:** A method of searching a sorted array using a divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers.
5. **Indexed Sequential Search:** In this searching method, first of all, an index file is created, that contains some specific group or division of required record when the index is obtained, then the partial indexing takes less time because it is located in a specified group.

2.12.1 Sequential Search [April 2

Sequential search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.

A simple approach is to do Sequential search, i.e.

- Start from the leftmost element of arr[] and one one compare x with each element of arr[]
- If x matches with an element, return the index.
- If x doesn't match with any of elements, return -1.

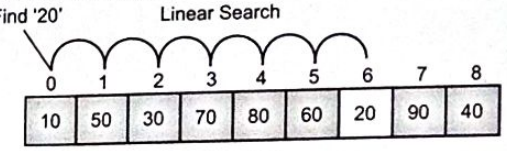


Fig. 2.14

Algorithm

Sequential Search (Array A, Value x)
Step 1: Set i to 1
Step 2: if i> n then go to step 7
Step 3: if A[i] = x then go to step 6
Step 4: Set i to i + 1
Step 5: Go to Step 2
Step 6: Print Element x Found at index i and go to step 8
Step 7: Print element not found
Step 8: Exit

Pseudocode

```
procedure Sequential_search (list, value)
for each item in the list
if match item == value
return the item's location
end if
end for
end procedure
```

2.12.2 Variant of Sequential Search- Sentinel Search

Sentinel Linear Search as the name suggests is a type Linear Search where the numbers of comparisons are reduc as compared to a traditional linear search. When a lin search is performed on an array of size N then in the wo case a total of N comparisons are made when the element be searched is compared to all the elements of the array a (N+1) comparisons are made for the index of the element be compared so that the index is not out of bounds of th array which can be reduced in a Sentinel Linear Search.

In this search, the last element of the array is replac with the element to be searched and then the linear search performed on the array without checking whether the curre index is inside the index range of the array or not because th element to be searched will definitely be found inside th

array even if it was not present in the original array since the ast element got replaced with it. So, the index to be checked will never be out of bounds of the array. The number of comparisons in the worst case here will be (N+2).

Examples:

Input: arr[] = {10, 20, 180, 30, 60, 50, 110, 100, 70}, x = 180

Output: 180 is present at index 2

Input: arr[] = {10, 20, 180, 30, 60, 50, 110, 100, 70}, x = 90

Output: Not found

Below is the implementation of the above approach:

Program 2.24: Sentinel Search.

```
def sentinel_search(arr, n, x):
    # Save the last element
    last = arr[n - 1]

    # Place the target at the last position
    arr[n - 1] = x
    i = 0
    # Perform the search
    while arr[i] != x:
        i += 1

    # Restore the original last element
    arr[n - 1] = last
    # Check if x was found within bounds or at last
    if i < n - 1 or arr[n - 1] == x:
        print(f"{x} is present at index {i}")
    else:
        print("Not found")

# Driver code
arr = [10, 20, 180, 30, 60, 50, 110, 100, 70]
n = len(arr)
x = 180

sentinel_search(arr, n, x)
```

Output:

180 is present at index 2

2.12.3 Binary Search [April 23]

Binary Search Requires Sorted Array

Steps to perform binary search on sorted array A of size n are as follows:

1. Initialize lower and upper limit of array to be searched Low=0, High = n–1.
2. Find the middle position

```
Mid=(Low + High)/2
```

3. Compare the input key value with the key value of the middle element of the array. It has 3 possibilities.

Case 1: If keys match then return mid and print element found.

```
if(key==A[mid])
return mid
```

Case 2: If key is less than middle element then element is present in left half of array. Update High and goto step 2.

```
if(key<A[mid])
High= Mid-1
goto step 2
```

Case 3: If key is greater than middle element then element is present in right half of array. Update Low and goto step 2.

```
if(key>A[mid])
Low= Mid+1
goto step 2
```

Case 4: if(Low>High)

Print element not found

Iterative algorithm for Binary Search

```
int BinarySearch( int A[], int x)
{
    int mid, n;
    int Low = 0;
    int High = n-1;
    while ( Low <= High )
    {
        mid = (Low + High) / 2;
        if ( A[mid] == x )
            return mid;
        if ( A[mid] > x )
            High = mid - 1;
        else Low = mid + 1;
    }
    return -1;
}
```

Example 2.3: *Let array A= {–1, 5, 6, 18, 19, 25, 46, 78, 102, 114}. We want to find element '6' using binary search.*

–1	5	6	18	19	25	46	78	102	114
0	1	2	3	4	5	6	7	8	9

Solution:

Step 1: Low = 0,

High = 9

Step 2: Mid = (Low + High)/2

Mid = (0 + 9) / 2 = 4.5 i.e. 4. It means middle element is located at position 4. So, A[Mid] is 19. Thus

Low = 0

Mid = 4

High = 9

Low			Mid						High
–1	5	6	18	19	25	46	78	102	114
0	1	2	3	4	5	6	7	8	9

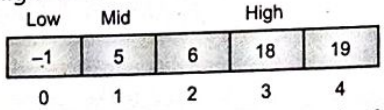
Step 3: Compare 6 with 19. It satisfies case 2 of step 3. so High = 4 – 1 = 3 and element is present in left half of array. We can ignore right half of array.

Repeat step 2. So mid= (0 + 3) / 2 = 1.5 i.e. 1. mid is at position 1. Element 5 is present at position 1.

Low = 0

Mid = 1

High = 3



Again compare 5 with 6. It satisfies case 3 of step 3. so

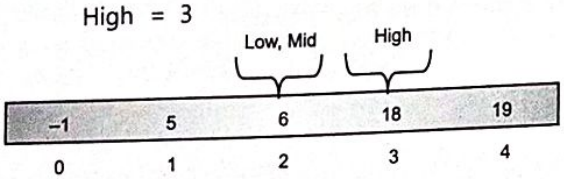
Low = 1 + 1 = 2.

Repeat step 2. Mid = (2 + 3) / 2= 2.5 i.e. 2. Mid is at position 2. Element 6 is at position 2.

Low = 2

Mid = 2

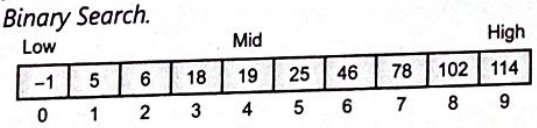
High = 3



Compare 6 with 6.

Match is found. Return the position and print element found.

Example 2.4: *Find 103 in {–1, 5, 6, 18, 19, 25, 46, 78, 102, 114} using Binary Search.*



Solution:

Step 1: Low = 0

High = 9

Step 2: Mid = (0 + 9) / 2 = 4

So middle element is 19.

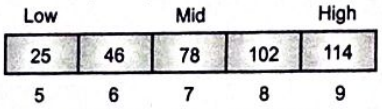
Step 3: Compare(103,19)

103>19 hence element is located at right half of array and we can ignore left half of array. Update Low and recalculate mid.

Low = 4 + 1 = 5

High = 9

Mid = (5 + 9) / 2 = 7



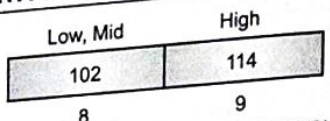
Middle element is 78. Compare(103,78)

103>78. Recalculate Low and Mid

Low = 7 + 1 = 8

High = 9

Mid = (8 + 9) / 2 = 8



Middle element is 102. Compare(103,102)

103>102. Recalculate Low and Mid.

Low = 8 + 1 = 9

High = 9

Here Low = High and match is not found hence we conclude that element is not present in array.

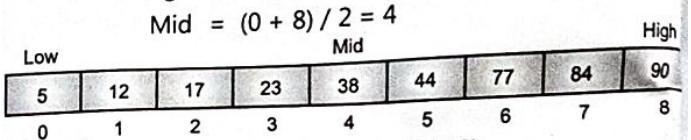
Example 2.5: *Find 44 in A={5,12,17,23,38,44,77,84,90} using binary search.*

Solution:

Step 1: Low = 0

High = 8

Mid = (0 + 8) / 2 = 4



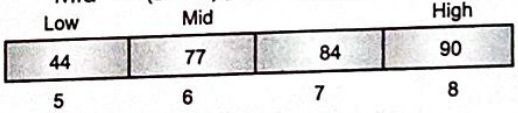
Middle element is 38. Compare (44,38)

44>38.Thus element is located at right half of array and we can ignore left half of array. Recalculate Low and Mid.

Low = (4 + 1) = 5

High = 8

Mid = (5 + 8) / 2 = 6.5 i.e. 6



Middle element is 77. Compare(44, 77)

44 < 77. Recalculate High and Mid.

Low = 5

High = 6 – 1 = 5

Mid = (5 + 5)/2 = 5



Middle element is 44. Compare(44,44)

Match is Found. Return index and print "Element Found."

Algorithm for Recursive Binary Search

```
int BinarySearch (int A[], int x, int Low, int High)
{
    if (High > Low)
        return -1;
int mid = (High + Low) / 2;
    if (A[mid] == x)
        return mid;
if (A[mid] < x)
        return BinarySearch(A, x, mid+1, High);
return BinarySearch(A, x, Low, mid-1);
}
```

Program 2.25: Binary Search.

```
    n = int(input("Enter the number of elements: "))
a = [0] * (n+1)

for i in range(1, n+1):
  a[i] = int(input(f"Enter the number in ascending order a[{i}]: "))

search = int(input("Enter the element to be searched: "))

low = 1
high = n
f = 0

while low <= high:
  mid = (low + high) // 2
  if search < a[mid]:
    high = mid - 1
  elif search > a[mid]:
    low = mid + 1
  else:
    f = 1
    print(f"Element is found at the position {mid}")
    break

if f == 0:
  print("Element not found")
```

Complexity Analysis of Binary Search

Assume our array size is 16.

When n = 16 BinarySearch is called to reduce size to n=8.

When n = 8 BinarySearch is called to reduce size to n=4.

When n = 4 BinarySearch is called to reduce size to n=2.

When n = 2 BinarySearch is called to reduce size to n=1.

Thus, we see that BinarySearch function is called 4 times i.e. 4 elements of the array were examined for n =16.

Note that $16 = 2^4$

Let us consider a more general case where n is still a power of 2. Let us say n =2k

After k searches, the while loop is executed k times and n reduces to size 1. Let us assume that each run of the while loop involves at most 4 operations.

Thus, total number of operations: 4k.

The value of k can be determined from the expression

$$2k = n$$

Taking log of both sides

$$k = \log n$$

Thus, total number of operations = 4 log n.

We conclude that the time complexity of the Binary search method is O(log n), which is much more efficient than the Sequential Search method.

Advantages

- Binary Search is fast and efficient.
- Suitable for large number of values.

Disadvantage

- Binary search works only on sorted array or list.

2.12.4 Fibonacci Search [April 23]

Given a sorted array arr[] of size n and an element x to be searched in it. Return index of x if it is present in array else return -1.

Examples:

Input: arr[] = {2, 3, 4, 10, 40}, x = 10

Output: 3

Element x is present at index 3.

Input: arr[] = {2, 3, 4, 10, 40}, x = 11

Output: -1

Element x is not present.

Fibonacci Search is a comparison-based technique that uses Fibonacci numbers to search an element in a sorted array.

Similarities with Binary Search:

1. Works for sorted arrays.
2. A Divide and Conquer Algorithm.
3. Has Log n time complexity.

Differences with Binary Search:

1. Fibonacci Search divides given array in unequal parts.
2. Binary Search uses division operator to divide range. Fibonacci Search doesn't use /, but uses + and -. The division operator may be costly on some CPUs.
3. Fibonacci Search examines relatively closer elements in subsequent steps. So, when input array is big that cannot fit in CPU cache or even in RAM, Fibonacci Search can be useful.
4. Fibonacci Numbers are recursively defined as F(n) = F(n-1) + F(n-2), F(0) = 0, F(1) = 1. First few Fibonacci Numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

Observations:

Below observation is used for range elimination, and hence for the O(log(n)) complexity.

F(n - 2) ≈ (1/3)*F(n) and

F(n - 1) ≈ (2/3)*F(n).

Algorithm:

Let the searched element be x.

The idea is to first find the smallest Fibonacci number that is greater than or equal to the length of given array. Let the found Fibonacci number be fib (m'th Fibonacci number). We use (m-2)'th Fibonacci number as the index (If it is a valid index). Let (m-2)'th Fibonacci Number be i, we compare arr[i] with x, if x is same, we return i. Else if x is greater, we recur for subarray after i, else we recur for subarray before i.

Below is the complete algorithm

Let arr[0..n-1] be the input array and element to be searched be x.

1. Find the smallest Fibonacci Number greater than or equal to n. Let this number be fibM [m'th Fibonacci Number]. Let the two Fibonacci numbers preceding it be fibMm1 [(m-1)'th Fibonacci Number] and fibMm2 [(m-2)'th Fibonacci Number].
2. While the array has elements to be inspected:

(i) Compare x with the last element of the range covered by fibMm2

(ii) If x matches, return index

(iii) Else If x is less than the element, move the three Fibonacci variables two Fibonacci down, indicating elimination of approximately rear two-third of the remaining array.

(iv) Else x is greater than the element, move the three Fibonacci variables one Fibonacci down. Reset offset to index. Together these indicate elimination of approximately front one-third of the remaining array.

Since there might be a single element remaining for comparison, check if fibMm1 is 1. If Yes, compare x with that remaining element. If match, return index.

Program 2.26: Fibonacci Search.

```
def fib_search(arr, x, n):
    # Initialize fibonacci numbers
    fibMMm2 = 0  # (m-2)'th Fibonacci
    fibMMm1 = 1  # (m-1)'th Fibonacci
    fibM = fibMMm2 + fibMMm1  # m'th Fibonacci

    # Find the smallest Fibonacci number >= n
    while fibM < n:
        fibMMm2 = fibMMm1
        fibMMm1 = fibM
        fibM = fibMMm2 + fibMMm1

    offset = -1

    while fibM > 1:
        # Check valid index
        i = min(offset + fibMMm2, n - 1)

        if arr[i] < x:
            fibM = fibMMm1
            fibMMm1 = fibMMm2
            fibMMm2 = fibM - fibMMm1
            offset = i
        elif arr[i] > x:
            fibM = fibMMm2
            fibMMm1 = fibMMm1 - fibMMm2
            fibMMm2 = fibM - fibMMm1
        else:
            return i  # Found

    # Check last element
    if fibMMm1 and offset + 1 < n and arr[offset + 1] == x:
        return offset + 1

    return -1  # Not found

# Driver code
arr = [10, 22, 35, 40, 45, 50, 80, 82, 85, 90, 100]
x = 85
n = len(arr)

index = fib_search(arr, x, n)
print(f"Found at index: {index}" if index != -1 else "Element not found")
```

Output:

Found at index: 8

Illustration:

Let us understand the algorithm with below example:

i	1	2	3	4	5	6	7	8	9	10	11	12
ar[i]	10	22	35	40	45	50	80	82	85	90	100	-

Illustration assumption: 1-based indexing. Target elem x is 85. Length of array n = 11.

Smallest Fibonacci number greater than or equal to 11 13. As per our illustration, fibMm2 = 5, fibMm1 = 8, a fibM = 13.

Another implementation detail is the offset variable (z initialized). It marks the range that has been eliminat starting from the front. We will update it time to time.

Now since the offset values is an index and all indic including it and below it has been eliminated, it only mak sense to add something to it. Since fibMm2 ma approximately one-third of our array, as well as the indices marks are sure to be valid ones, we can add fibMm2 to offs and check the element at index i = min(offset + fibMm2, n).

fibMm2	fibmm 1	fibM	offset	i=min(offset+ fibLn)	arr[i]	Consequence
5	8	13	0	5	45	Move one down, reset offset
3	5	8	5	8	82	Move one down, reset offset
2	3	5	8	10	90	Move two down
1	1	2	8	9	85	Return i

Visualization:

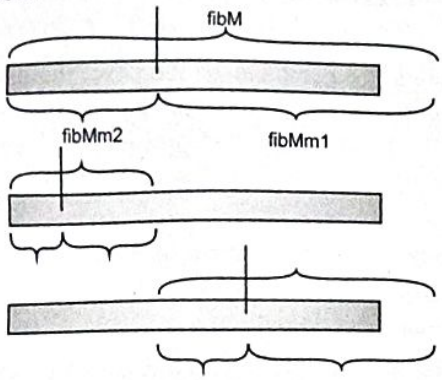


Fig. 2.15

2.12.5 Indexed Sequential Search [April 23]

In this searching method, first of all, an index file is created, that contains some specific group or division of required record when the index is obtained, then the partial indexing takes less time because it is located in a specified group.

Note: When the user makes a request for specific records it will find that index group first where that specific record is recorded.

Characteristics of Indexed Sequential Search:

- In Indexed Sequential Search a sorted index is set aside in addition to the array.
- Each element in the index points to a block of elements in the array or another expanded index.
- The index is searched 1st then the array and guides the search in the array.

Note: Indexed Sequential Search actually does the indexing multiple time, like creating the index of an index.

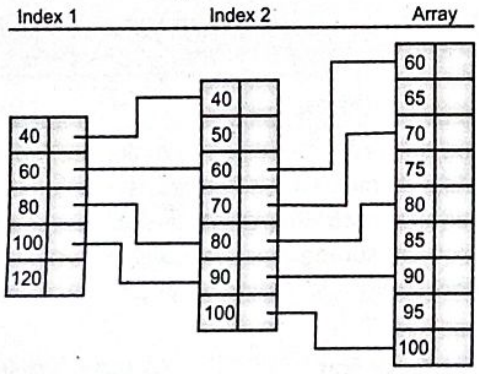


Fig. 2.16: Indexed sequential search

Program 2.27: Sequential Search.

```
def indexed_sequential_search(arr, k):
    elements = []
    indices = []
    n = len(arr)
    # Create index blocks (every 3rd element)
    for i in range(0, n, 3):
        elements.append(arr[i])
        indices.append(i)
    # Check if k is less than the first index block value
    if k < elements[0]:
        print("Not found")
        return
    # Find block where k could be
    start = 0
    end = n - 1
    for i in range(1, len(elements)):
        if k < elements[i]:
            start = indices[i - 1]
            end = indices[i]
            break
    # Linear search in that block
    for i in range(start, end + 1):
        if i >= n:
            break
        if arr[i] == k:
            print(f"Found at index {i}")
            return
    print("Not found")
# Driver code
arr = [6, 7, 8, 9, 10]
k = 8
indexed_sequential_search(arr, k)
```

Output:

Found at index 2

2.13 SORTING [Oct. 22, April 23]

2.13.1 Concepts of Sorting

Sorting is a fundamental operation in computer science and data processing that arranges data in a specified order either ascending or descending. Sorting plays a crucial role in searching, data analysis, and algorithm optimization. To evaluate and compare different sorting algorithms, we use several important characteristics:

1. Stability of Sorting Algorithms

Definition:

A sorting algorithm is said to be stable if it preserves the relative order of records with equal keys (values).

Example:

Consider the following list of student records sorted by marks (where the second field is name):

```
scss
CopyEdit
(90, Alice), (90, Bob), (85, Charlie)
```

If a stable sorting algorithm is used, after sorting by marks:

```
scss
CopyEdit
(85, Charlie), (90, Alice), (90, Bob)
```

The relative order of Alice and Bob is maintained, as they both had the same mark.

Importance of Stability:

- Essential in multi-level sorting (e.g., sorting first by name, then by age).
- Useful in applications involving records, tuples, or objects with multiple fields.

Stable Sorting Algorithms:

- Bubble Sort
- Insertion Sort
- Merge Sort
- Tim Sort (used in Python and Java)

Unstable Sorting Algorithms:

- Selection Sort
- Quick Sort (unless modified)
- Heap Sort

2. Efficiency of Sorting Algorithms

Definition:

Efficiency of a sorting algorithm refers to how well it performs in terms of time and space complexity, especially as the size of the input data grows.

Time Complexity:

- **Best Case:** Minimum steps needed (e.g., already sorted).
- **Average Case:** Expected number of steps over random data.
- **Worst Case:** Maximum steps required (e.g., reverse sorted).

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Heap Sort	O(n log n)	O(n log n)	O(n log n)

Space Complexity:

- Some algorithms use additional memory (e.g., Merge Sort).
- Others sort in-place (e.g., Quick Sort, Heap Sort).

Efficiency Considerations:

- For large datasets, O(n log n) algorithms are preferred.
- For small or nearly sorted datasets, insertion sort may be more efficient.

3. Number of Passes

Definition:

A pass refers to a complete iteration over the data du the sorting process. The number of passes indicates h many times the sorting algorithm processes the entire o part of the dataset to achieve the final sorted order.

Example:

- **Bubble Sort:** In the worst case, it makes *n-1 passes*
- **Selection Sort:** Also makes *n-1 passes*, selecting minimum each time.
- **Insertion Sort:** Requires up to *n-1 passes* in the wo case.
- **Merge Sort:** Works in a recursive manner, with lo levels (or passes) during merge phase.
- **Radix Sort:** Number of passes depends on number of digits (d), it performs *d passes*.

Why it Matters?

- More passes typically mean higher processing time
- External sorting (e.g., large datasets on disk) sensitive to the number of passes.
- Algorithms like Radix Sort or Counting Sort depe heavily on pass count for performance.

2.13.2 Internal Sorting [April

The internal sorting methods are applied to sm collection of data. As the entire collection of data to be sor in internal sort is small enough, sorting can take place wit main memory of a computer. Internal Sorting algorithms of 7 types:

1. Insertion Sort
2. Selection Sort
3. Merge Sort
4. Radix Sort
5. Quick Sort
6. Heap Sort
7. Bubble Sort

2.13.3 External Sorting [April

The External sorting methods are applied when number of data elements to be sorted is too large. Th methods involve as much external processing as processing the CPU. Hence this sorting requires auxiliary storage such disk or tapes. Algorithms for External sort are

1. Merge Sort
2. Replacement Sort

2.13.4 Bubble Sort [Oct. 22, April

Steps to perform Bubble sort are as follows.

Assume we want to sort the elements in ascending or and no two elements are equal.

1. Compare first element with second element. If f element is greater than next element, we interchan their position accordingly. i.e. first element is mov to second element's position and second element moved to first element's position. If No, then don't interchange any elements.

2. The element in second position is compared with element in third position and the process of interchanging elements is performed if second is greater than third element. The whole process of comparing and interchanging is repeated till last element. When the process gets completed, the largest element in array will get placed in the last position of the array.

Let us consider array A of size n. Then Algorithm for Bubble sort is

1. Set i to 0.
2. Set j to 0.
3. if(A[j]>A[j+1])
   swap A[j],A[j+1]
4. Increment j
5. if j<(n-1-i) goto step 3
6. Increment i
7. if(i<n-1) goto step 2.

Example 1: Suppose the following numbers are stored in an array A:

32	51	27	85	66	23	13	57
j = 0	1	2	3	4	5	6	7

We apply bubble sort to array A. We discuss each pass separately.

Pass 1:

1. Compare j[0] and j[1]. Since 32 < 51, the list is not altered.
2. Compare j[1] and j[2] Since 51 > 27, interchange 51 and 27 as follows:

32	51	27	85	66	23	13	57
j = 0	1	2	3	4	5	6	7

Array A becomes

32	27	51	85	66	23	13	57
j = 0	1	2	3	4	5	6	7

3. Compare j[2] and j[3]. Since 51 < 85, array is not altered.
4. Compare j[3] and j[4]. Since 85 > 66, interchange 85 and 66 as follows:

32	27	51	85	66	23	13	57
j = 0	1	2	3	4	5	6	7

Array A becomes

32	27	51	66	85	23	13	57
j = 0	1	2	3	4	5	6	7

5. Compare j[4] and j[5]. Since 85 > 23, interchange 85 and 23 as follows:

32	27	51	66	85	23	13	57
j = 0	1	2	3	4	5	6	7

Array A becomes

32	27	51	66	23	85	13	57
j = 0	1	2	3	4	5	6	7

6. Compare j[5] and j[6]. Since 85 > 13, interchange 85 and 13 as follows:

32	27	51	66	23	85	13	57
j = 0	1	2	3	4	5	6	7

32	27	51	66	23	13	85	57
j = 0	1	2	3	4	5	6	7

7. Compare j[6] and j[7]. Since 85 > 57, interchange 85 and 57 as follows:

32	27	51	66	23	13	85	57
j = 0	1	2	3	4	5	6	7

Array A becomes

32	27	51	66	23	13	57	85
j = 0	1	2	3	4	5	6	7

At the end of this first pass, the largest number, 85, has moved to the last position. However, the rest of the numbers are not sorted, even though some of them have changed their positions.

Pass 2 will move second last number at second last position, Pass 3 will move third last number at third last position and so on. Here we show array after every pass.

Pass 2:

27	33	51	23	13	57	66	85
j = 0	1	2	3	4	5	6	7

Pass 3:

27	33	23	13	51	57	66	85
j = 0	1	2	3	4	5	6	7

Pass 4:

27	23	13	33	51	57	66	85
j = 0	1	2	3	4	5	6	7

Pass 5:

23	13	27	33	51	57	66	85
j = 0	1	2	3	4	5	6	7

Pass 6:

13	23	27	33	51	57	66	85
j = 0	1	2	3	4	5	6	7

Hence the array is sorted in 6 passes.

Example 2: Apply Bubble Sort on array A= {5,3,1,9,8,2,4,7}.

j=0	5	3	1	9	8	2	4	7
j=1	3	5	1	9	8	2	4	7
j=2	3	1	5	9	8	2	4	7
j=3	3	1	5	9	8	2	4	7
j=4	3	1	5	8	9	2	4	7
j=5	3	1	5	8	2	9	4	7
j=6	3	1	5	8	2	4	9	7
j=7	3	1	5	8	2	4	7	9

Pass 2: i=2

3	1	5	8	2	4	7	9
1	3	5	8	2	4	7	9
1	3	5	8	2	4	7	9
1	3	5	8	2	4	7	9
1	3	5	2	8	4	7	9
1	3	5	2	4	8	7	9
1	3	5	2	4	7	8	9

Pass 3:

1	3	5	2	4	7	8	9
1	3	5	2	4	7	8	9
1	3	5	2	4	7	8	9
1	3	2	5	4	7	8	9
1	3	2	4	5	7	8	9

Pass 4:

1	3	2	4	5	7	8	9
1	3	2	4	5	7	8	9
1	2	3	4	5	7	8	9

This array gets sorted in 4 passes only.

Example 3: Apply Bubble Sort on array

A = {23, 78, 45, 8, 32, 56}

Pass 1:

23	78	45	8	32	56
23	78	45	8	32	56
23	45	78	8	32	56
23	45	8	78	32	56
23	45	8	32	78	56
23	45	8	32	56	78

Pass 2:

23	45	8	32	56	78
23	45	8	32	56	78
23	8	45	32	56	78
23	8	32	45	56	78

Pass 3:

23	8	32	45	56	78
8	23	32	45	56	78

This array is sorted in 3 passes.

Program 2.28: Write a Program for Bubble Sort in C.

```
def bubble_sort(A, n):
    for i in range(n - 1):
        for j in range(n - i - 1):
            if A[j] > A[j + 1]:
                # Swap
                A[j], A[j + 1] = A[j + 1], A[j]
# Main execution
A = []
n = int(input("Enter number of elements: "))
print("Enter the elements:")
for i in range(n):
    A.append(int(input()))

bubble_sort(A, n)

print("Sorted list in ascending order:")
for num in A:
    print(num)
```

Complexity Bubble Sort:

Suppose we have a list with n elements, and each elem perform n–1 comparison with elements to its left, and swa necessary.

Best Case: If the list is already sorted, only one iteratio performed and complexity is O(1).

Average and Worst Case: For n elements at most swap operation is performed in each pass. The worst average case complexity is $O(n^2)$.

2.13.5 Selection Sort [Oct. 22, April

Assume we want to sort list in ascending order. Selecti sort finds the smallest element from unsorted list and swa with the element in first position. Then it finds second sm element and swap it with element at second position. T process is repeated till entire list is sorted in ascending or Each time we swap elements, we say that we have comple a sort pass. A list of n elements requires n–1 passes completely rearrange the data.

Procedure for every pass is as follows.

Pass 1: Find the position P of the smallest in the list elements A[l], A[2], . . . , A[N], and then interchange A[P] A[1]. Then A[1] is sorted.

Pass 2: Find the position P of the smallest in the sublist N –1 elements A[2], A[3],. . . , A[N], and then interchan A[P]and A[2]. Then A[l], A[2] is sorted.

Pass 3: Find the position P of the smallest in the sublist N–2 elements A[3], A[4], . . . , A[N], and then interchange A and A[3]. Then: A[l], A[2] , A[3] is sorted.

Pass N–1: Find the position P of the smaller of elements A[N –1), A[N], and then interchange A[P] and A[N Then: A[l], A[2], . . . , A[N] is sorted. Thus, A is sorted after N passes.

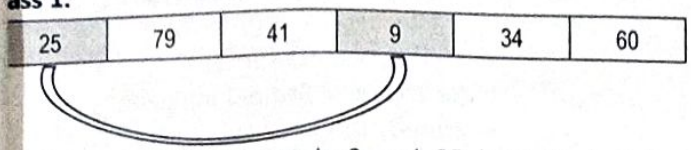
Example 1 :

25	79	41	9	34	60

Fig. 2.17: Original List

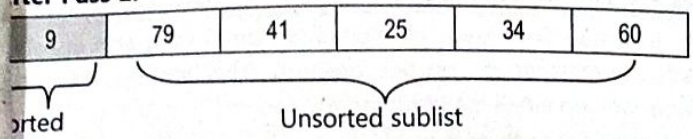
We will apply selection sort on this list.

ass 1:



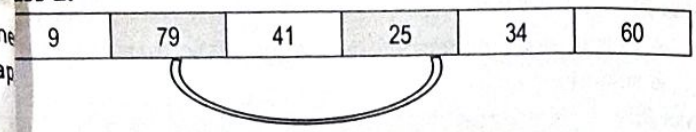
Here smallest element is 9 and 25 is located at first osition. So, we swap 9 with 25.

fter Pass 1:



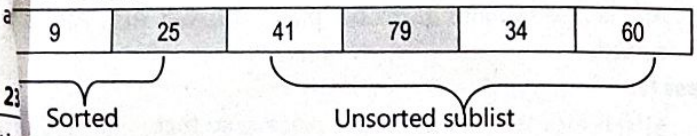
Now we find smallest element from unsorted sublist and ace it in second position.

ass 2:

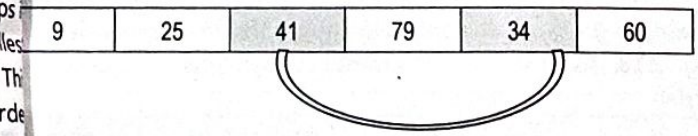


Unsorted sublist has 25 as a smallest element and 79 is cated at second position. Hence, we swap 25 and 79.

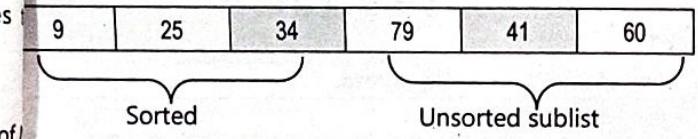
fter Pass 2:



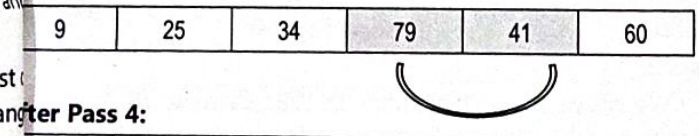
ss 3:



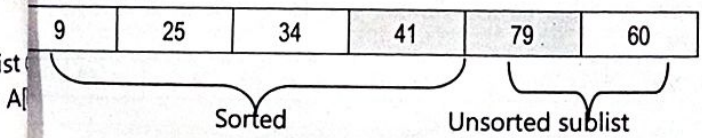
ter Pass 3:



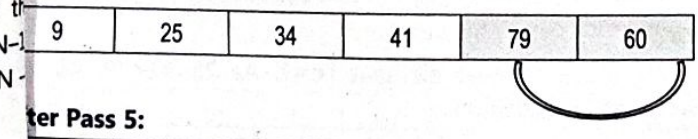
ass 4:



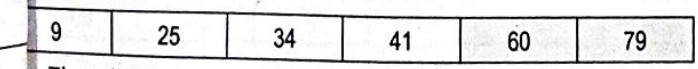
ter Pass 4:



ss 5:



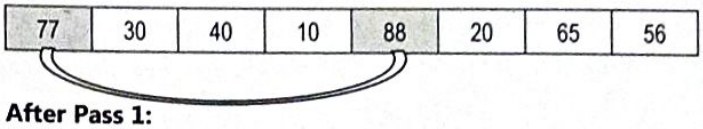
ter Pass 5:



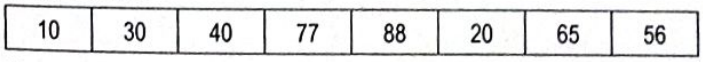
Thus, list is sorted.

Example 2: Apply selection sort on array A = {77, 30, 40, 10, 88, 20, 65, 56}

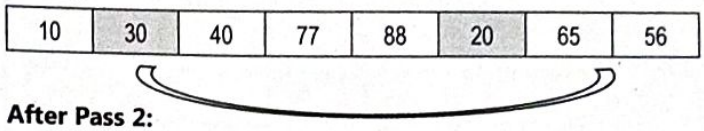
Pass 1:



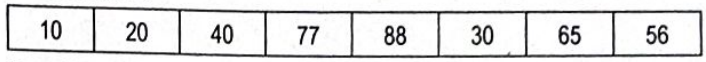
After Pass 1:



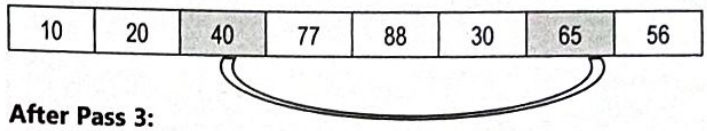
Pass 2:



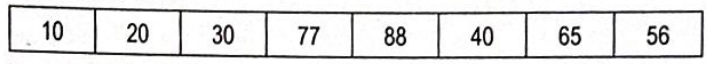
After Pass 2:



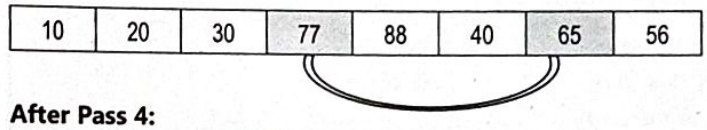
Pass 3:



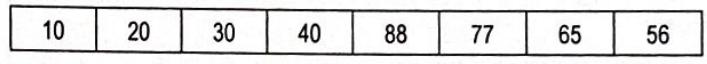
After Pass 3:



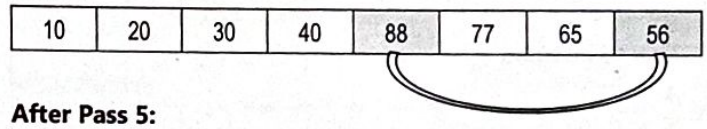
Pass 4:



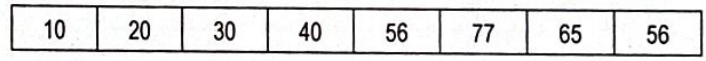
After Pass 4:



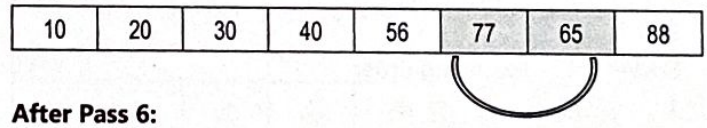
Pass 5:



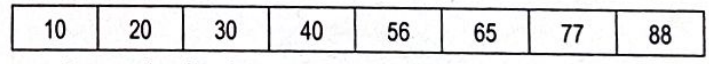
After Pass 5:



Pass 6:



After Pass 6:



Hence the list is sorted in ascending order by applying selection sort.

Program 2.29: Selection Sort.

```
def selection_sort(A, n):
    for i in range(n - 1):
        s = i
        for j in range(i + 1, n):
            if A[j] < A[s]:
                s = j
        if s != i:
            A[i], A[s] = A[s], A[i]
```

Program to Perform Selection Sort.

```
# Selection Sort in Python

# Input number of elements
n = int(input("Enter number of elements: "))

# Input array elements
print(f"Enter {n} integers:")
A = []
for i in range(n):
    A.append(int(input()))

# Selection Sort
for i in range(n - 1):
    s = i
    for j in range(i + 1, n):
        if A[s] > A[j]:
            s = j
    if s != i:
        A[i], A[s] = A[s], A[i]

# Output sorted array
print("Sorted list in ascending order:")
for num in A:
    print(num)
```

Output:

```
Enter number of elements
5
Enter 5 integers
4
1
7
5
9
Sorted list in ascending order
1
4
5
7
9
```

Complexity of Selection Sort:

In selection sort outer for loop is executed n–1 times. On the kth time through the outer loop, initially the sorted list holds k–1 elements and unsorted portion holds n–k+1 elements. In inner for loop 2 elements are compared each time.

Thus, 2*(n–k) elements are examined by the inner loop during the kth pass through the outer loop. But k ranges from 1 to n–1.

Total number of elements examined is:

$$\begin{aligned} T(n) &= 2*(n-1) + 2*(n-2) + 2*(n-3) + \ldots \\ &\quad + 2*(n-(n-2)) + 2*(n-(n-1)) \\ &= 2*((n-1) + (n-2) + (n-3) + \ldots + 2 + 1) \\ &\quad \text{(or } 2*\text{(sum of first n–1 integers)} \\ &= 2*((n-1)*n)/2) \\ &= n^2 - n, \text{ so complexity of algorithm is} \\ &\quad O(n^2). \end{aligned}$$

2.13.6 Insertion Sort [Oct. 22, April

Insertion Sort reads all elements from 1 to n and ins each element at its proper position. This sorting met works well on small list of elements.

Procedure for each pass is as follows:

Pass 1:

A[l] by itself is trivially sorted.

Pass 2:

A[2] is inserted either before or after A[l] so that: A[l], is sorted.

Pass 3:

A[3] is inserted into its proper place so that: A[l], A[2], is sorted.

Pass 4:

A[4] is inserted into its proper place A[l], A[2], A[3], A[sorted.

Pass N:

A[N] is inserted into its proper place in so that A[l], A[2], ..., A[N] is sorted

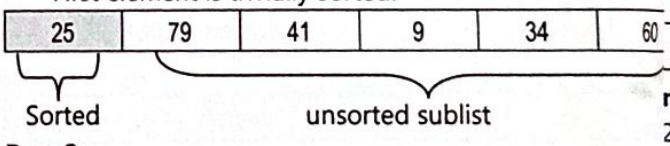
Example 1: We will apply insertion sort on original li Fig. 2.18. Assume we are arranging elements in ascen order.

25	79	41	9	34	60

Fig. 2.18: Original list

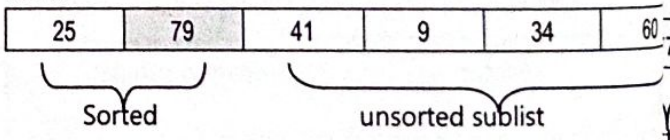
Pass 1:

First element is trivially sorted.



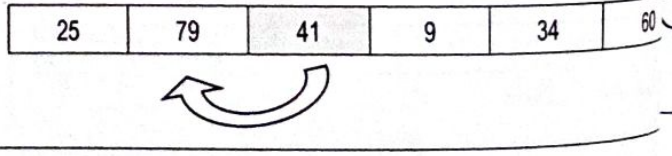
Pass 2:

We move to next element i.e. 79. Element 79 is at proper position as 25<79. So, we do not alter the pos of 79.

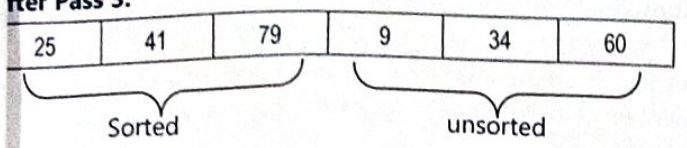


Pass 3:

We move to next element i.e.41. As 25<41<79, 4 inserted at 2nd position.

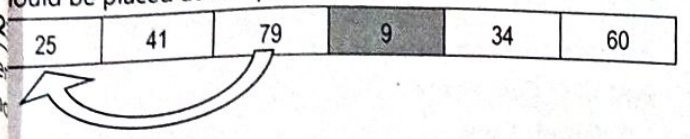


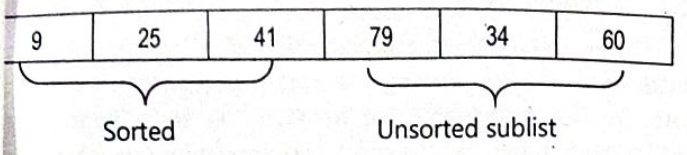
fter Pass 3:



ass 4:

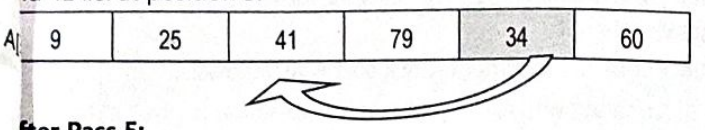
We move to next element i.e. 9. Element 9 is smallest so it ould be placed at first position.



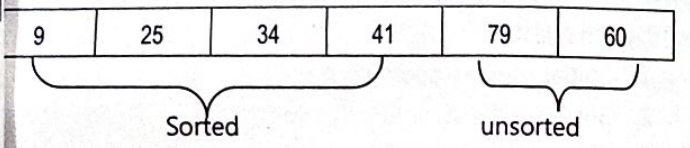


ass 5:

Consider next element 34. It should be placed between 25 nd 41 i.e. at position 3.

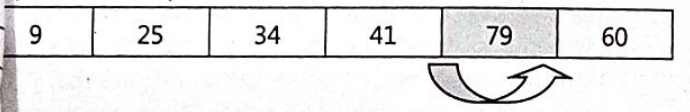


fter Pass 5:

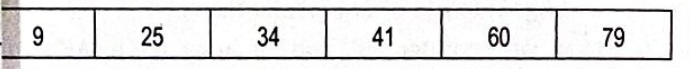


ss 6:

Next element is 79. It is the largest element and it should placed at last position.



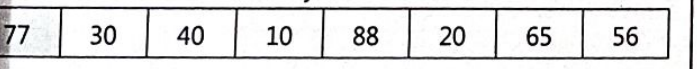
fter Pass 6:

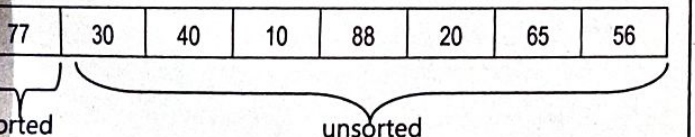


Hence the list is sorted using insertion sort.

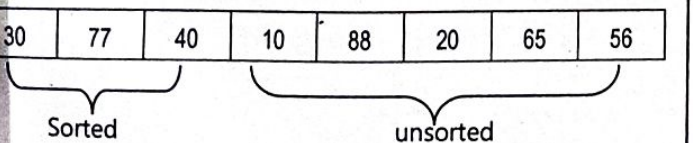
ample 2: Apply Insertion sort on array A = {77, 30, 40, 10, , 20, 65, 56}

ss 1: First element is trivially sorted.

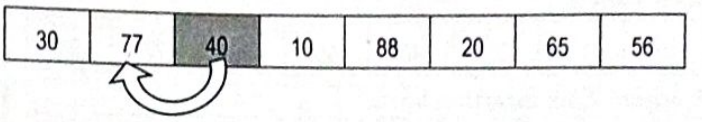




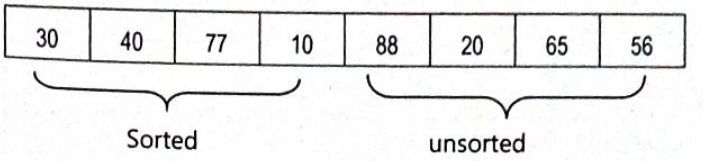
ss 2:



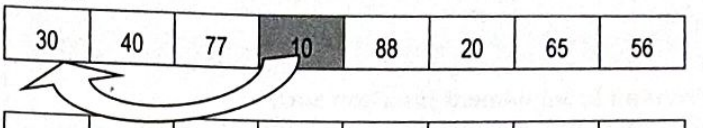
Pass 3:

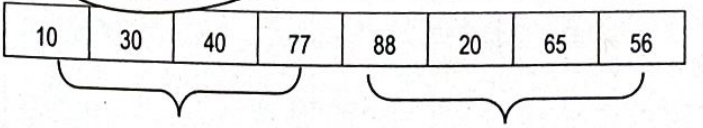


After Pass 3:



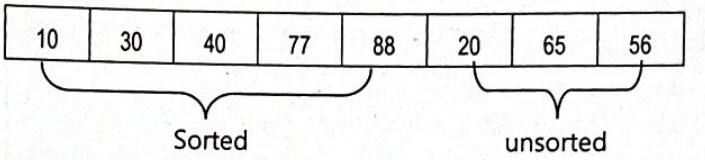
Pass 4:



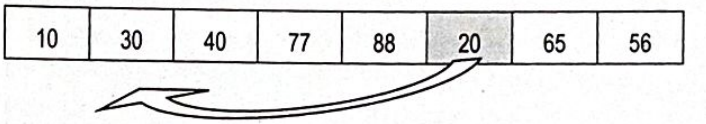




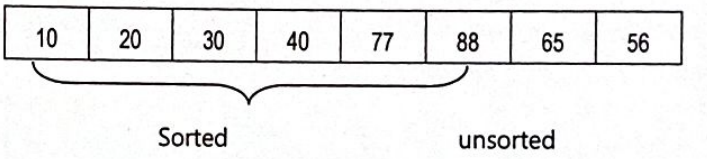
Pass 5:



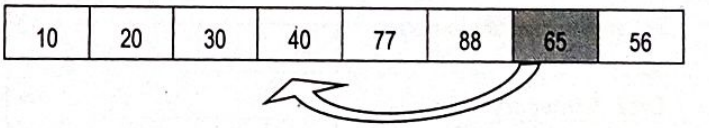
Pass 6:



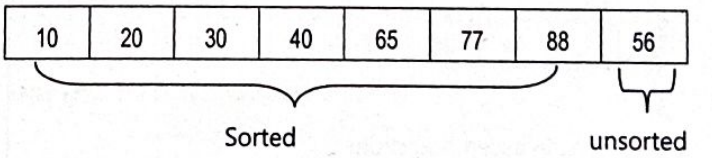
After Pass 6:



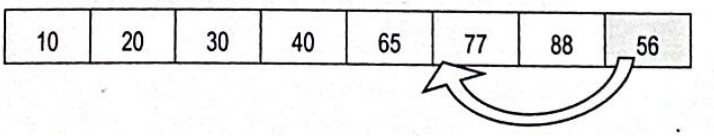
Pass 7:



After Pass 7:



Pass 8:



After Pass 8:

10	20	30	40	56	65	77	88

Program 2.30: Insertion Sort.

```
def insertion_sort(A, n):
    for i in range(1, n):
        temp = A[i]
        j = i - 1
        while j >= 0 and A[j] > temp:
            A[j + 1] = A[j]
            j -= 1
        A[j + 1] = temp
```

Program to Implement Insertion Sort.

```
# Insertion Sort in Python
# Input number of elements
n = int(input("Enter number of elements: "))

# Input array elements
print(f"Enter {n} integers:")
A = []
for i in range(n):
    A.append(int(input()))
# Insertion Sort Logic
for i in range(1, n):
    j = i
    while j > 0 and A[j] < A[j - 1]:
        # Swap A[j] and A[j-1]
        A[j], A[j - 1] = A[j - 1], A[j]
        j -= 1

# Output sorted array
print("Sorted list in ascending order:")
for num in A:
    print(num)
```

Output:

```
Enter number of elements
5
Enter 5 integers
9
4
2
5
3
Sorted list in ascending order
2
3
4
5
9
```

Complexity of Insertion Sort:

Worst Case Complexity: In the worst case, for each do (i – 1) swaps inside the inner for loop therefore, o number of swaps (when i goes from 2 to n in the out loop) is

$$T(n) = 1 + 2 + 3 + \ldots + (I-1) + \ldots + n-1$$
$$= n \times (n-1)/2$$
$$= O(n^2)$$

Average case Complexity is $O(n^2)$.

Best Case Complexity is O(n).

2.13.7 Quick Sort [Oct. 22, Apr

Quick sort is most efficient method of sorting. It divide and conquer strategy to sort the elements. In sort, we divide the array of items to be sorted into partitions and then call the quick sort procedure recursive sort the two partitions. For this we choose a pivot ele and arrange all elements such that all the elements in th part are less than the pivot and all those in the right par greater than it.

Algorithm to implement quick sort is as follows:

Consider array A of size n to be sorted. p is pivot elem Index positions of first and last element of array A is den by 'first' and 'last'.

1. Initially first=0 and last=n–1.
2. Increment first till A[first]<=p
3. Decrement last till A[last]>=p
4. If first < last then swap A[first] with A[last].
5. If first > last then swap p with A[last] and t position of p is found. Pivot p is placed such tha elements to its left are less than it and all eleme right to it are greater than it. Let us assume tha final position of p. Then A[0] to A[j–1] are less th and A[j+1] to A[n–1] are greater than p.
6. Now we consider left part of array A i.e. A[0 A[j–1]. Repeat step 1,2,3 and 4. Apply same proce for right part of array.
7. We continue with above steps till no more parti can be made for every subarray.

C Function for Quick Sort

```
def quicksort(arr, first, last):
    if first < last:
        pivot = first
        i = first
        j = last

        while i < j:
            while i < last and arr[i] <= arr[pivot]:
                i += 1
            while arr[j] > arr[pivot]:
                j -= 1
```

```
    if i < j:
        arr[i], arr[j] = arr[j], arr[i]

    arr[pivot], arr[j] = arr[j], arr[pivot]

    quicksort(arr, first, j - 1)
    quicksort(arr, j + 1, last)
```

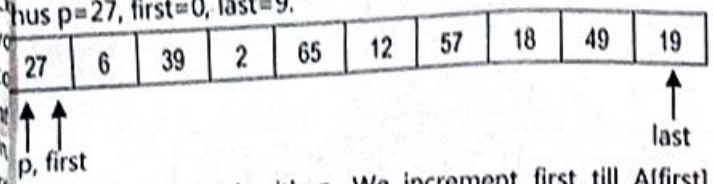
We will see examples on Quick sort

Example 1: Apply Quick sort on array

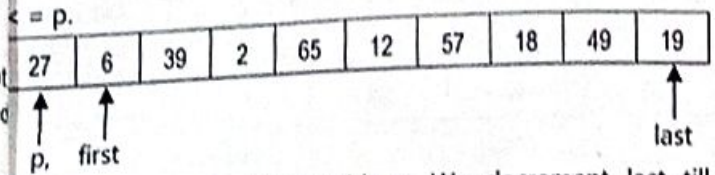
A = {27, 6, 39, 2, 65, 12, 57, 18, 49, 19}

Pass 1:

Generally, first element is considered as a pivot element. Thus p=27, first=0, last=9.



Compare A[first] with p. We increment first till A[first] <= p.

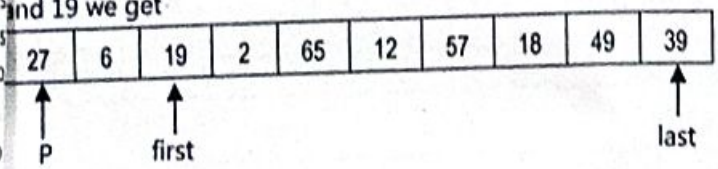


Now, compare A[last] with p. We decrement last till A[last]>=p.

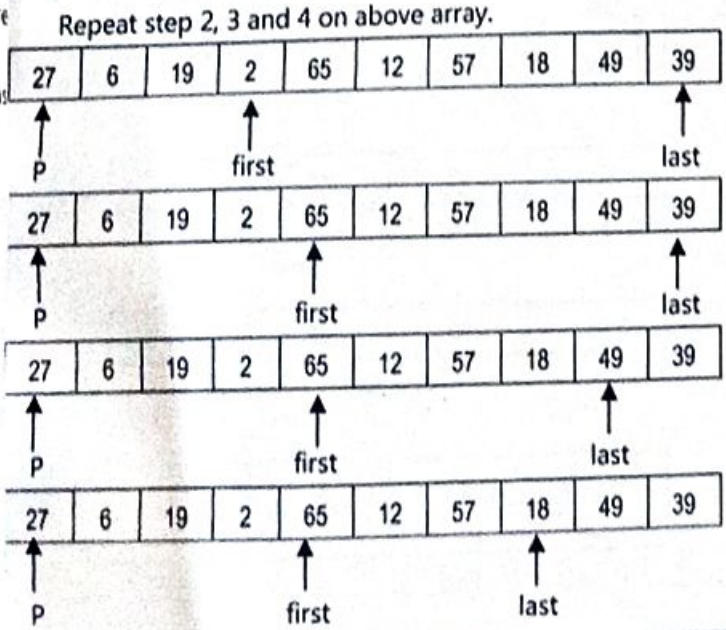
Here 19 < 27. So, condition A[last]>=p becomes false and we stop decrementing last.

Currently first=2 and last=9.

i.e. first < last so we swap A[first] and A[last].Swapping 39 and 19 we get

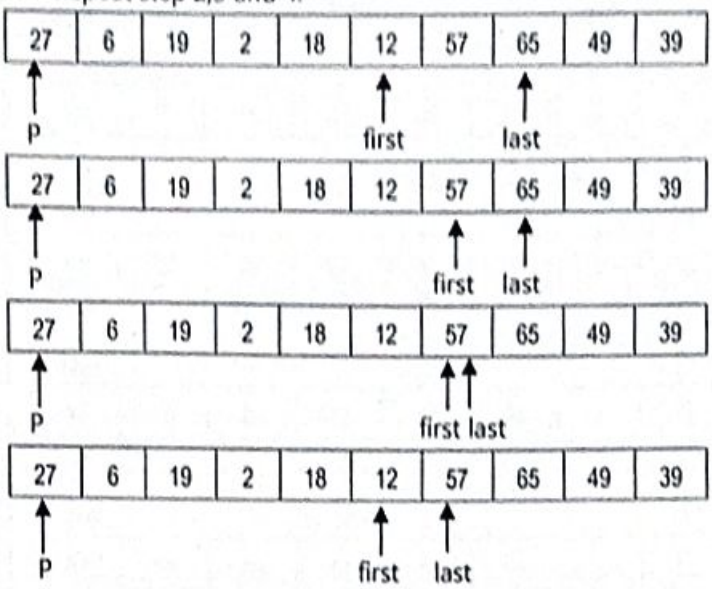


Repeat step 2, 3 and 4 on above array.

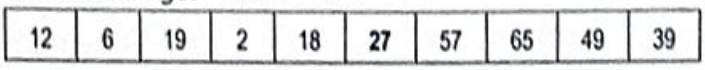


Here again first<last so we swap A[first] and A[last]. Swapping 65 and 18 we get

Repeat step 2,3 and 4.



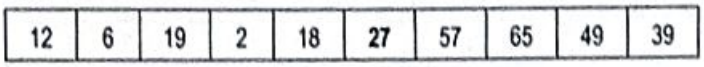
Now First>last so we swap pivot p with A[last]. Swapping 27 and 12 we get



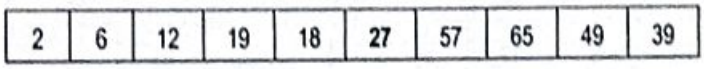
Here we complete Pass 1. Now pivot element is 27 and it divides array in left and right partition. We can see that elements less than pivot are in left partition and elements greater than pivot are in right partition.

Now we continue same procedure for left and right partition. Here we see results of corresponding passes.

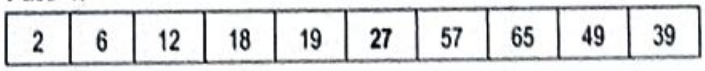
Pass 2:



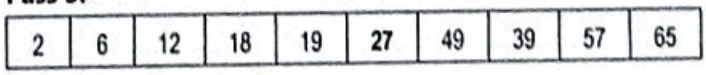
Pass 3:



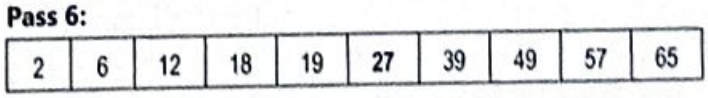
Pass 4:



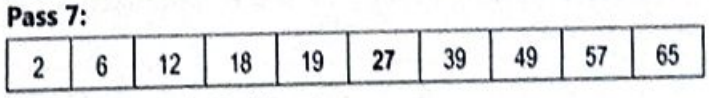
Pass 5:



Pass 6:



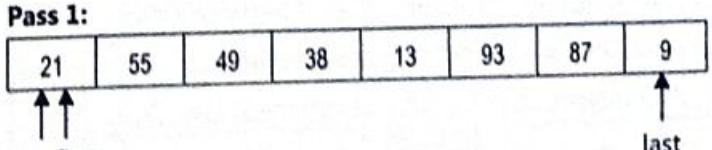
Pass 7:

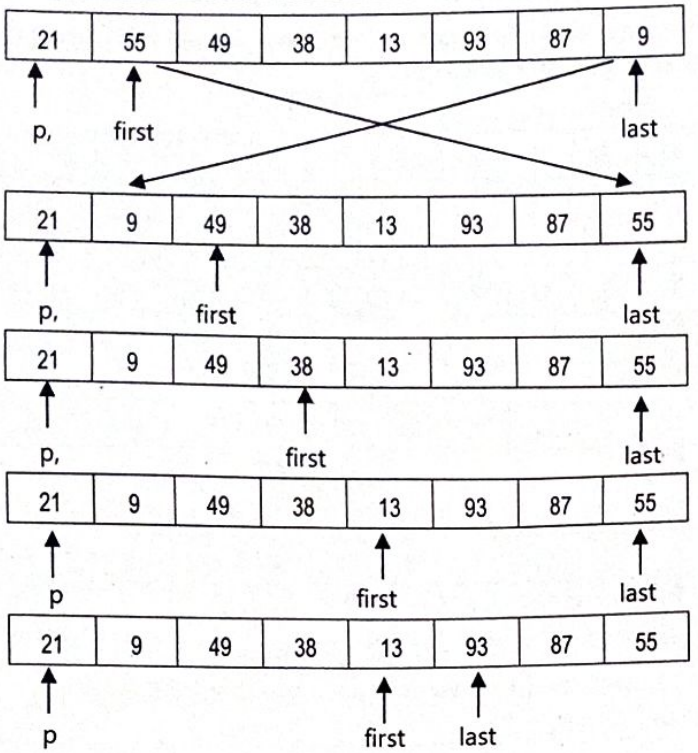


Example 2: Apply Quicksort on array

A = {21, 55, 49, 38, 13, 93, 87, 9}

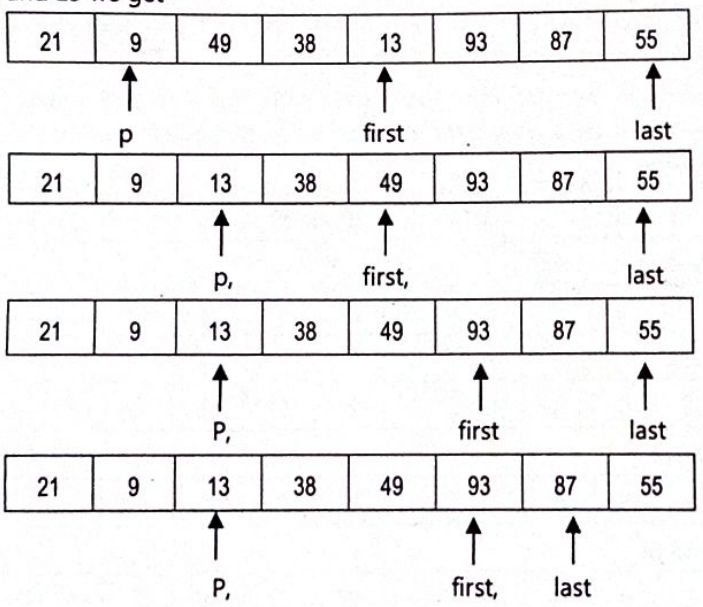
Pass 1:



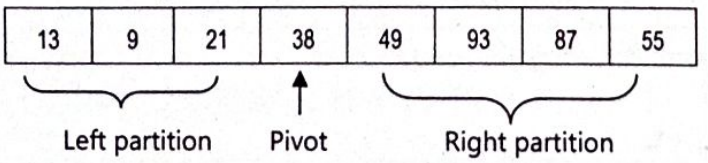


Pass 2:

Now first>last hence swap pivot and A[last].Swapping 21 and 13 we get

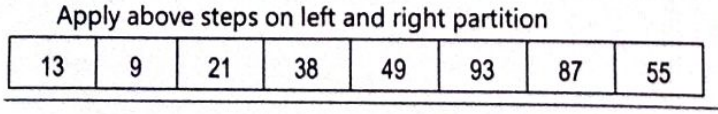


Now first > last hence swap pivot and A [last]'. Swapping 21 and 13 we get

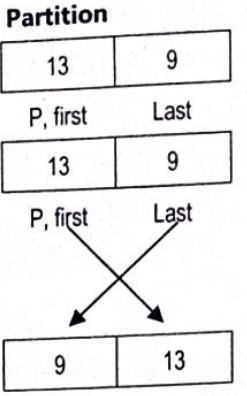


Apply above steps on left and right partition

Apply above steps on left and right partition

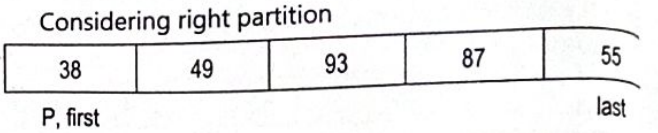


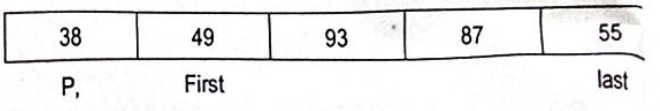
Left Partition

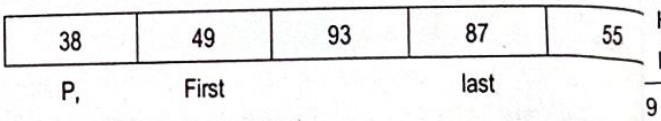


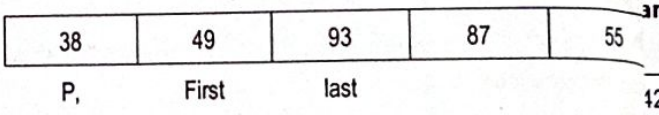
Hence the left partition is sorted.

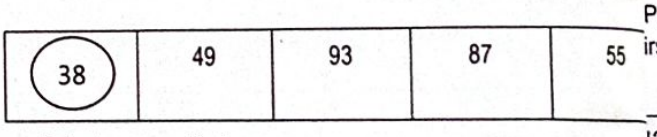
Considering right partition

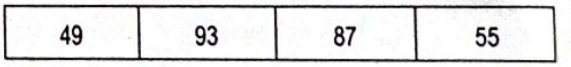


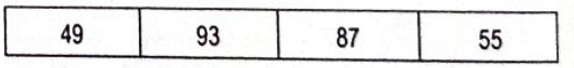


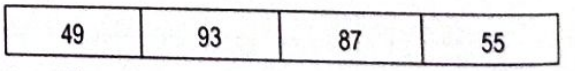


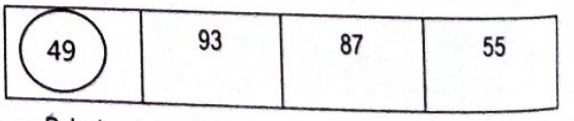


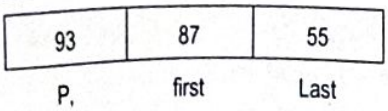


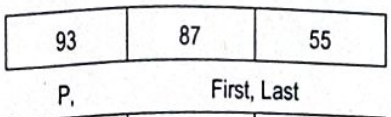


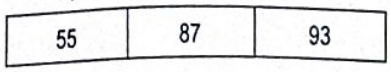


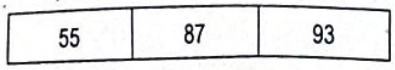


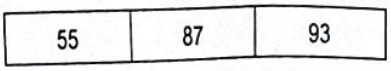


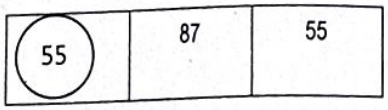






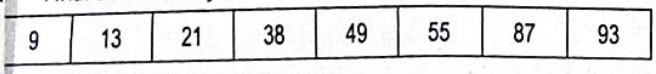






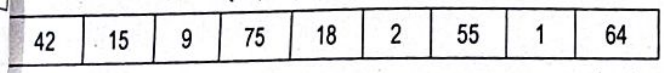
Hence the right partition is sorted.

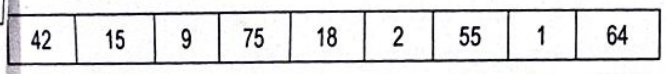
Final sorted array is

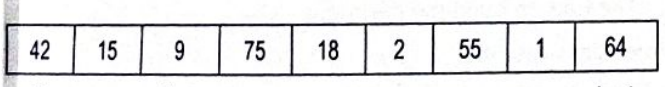


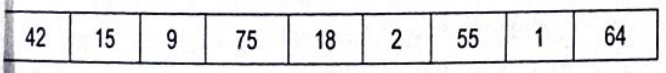
Example 3: Apply Quick Sort on

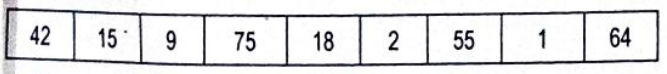
A = {42, 15, 9, 75, 18, 2, 55, 1, 64}

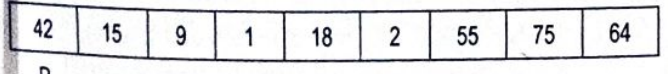


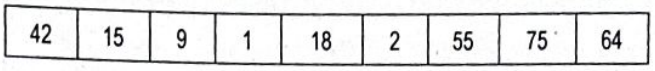


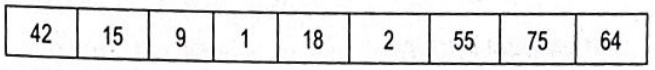


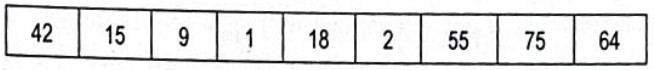


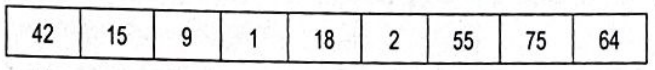


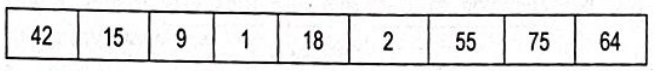




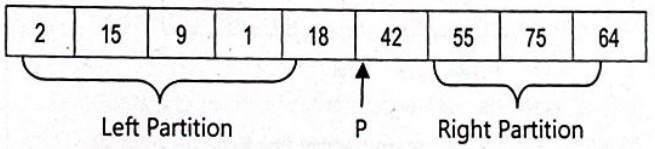




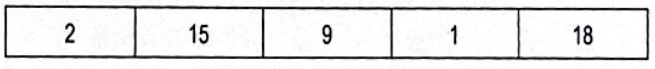


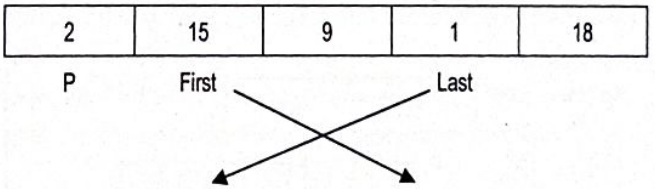


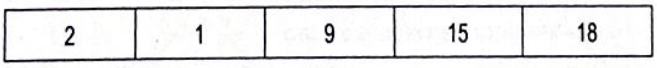
Swap p and A[last]

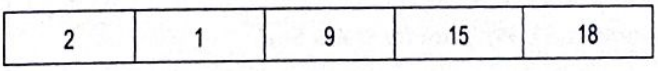


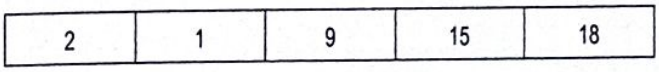
Applying Quick Sort on Left Partition











2	1	9	15	18

P Last First

As first>last swap p and A[last]

1	2	9	15	18

Left partition is sorted.

Apply Quick sort on right partition

55	75	64

P, First last

55	75	64

P First last

55	75	64

P First, last

55	75	64

P, last First

75	64

P, First Last

75	64

P First, last

75	64

P, last First

64	75

Hence the right partition is sorted.

Sorted array is

1	2	9	15	18	42	55	64	75

Program 2.31: Program for Quick Sort.

```
def quicksort(A, first, last):
  if first < last:
    pivot = first
    i = first
    j = last
    while i < j:
      while i < last and A[i] <= A[pivot]:
        i += 1
      while A[j] > A[pivot]:
        j -= 1
      if i < j:
        A[i], A[j] = A[j], A[i]
    A[pivot], A[j] = A[j], A[pivot]
    quicksort(A, first, j - 1)
    quicksort(A, j + 1, last)
# Main code
A = []
n = int(input("Enter size of the array: "))
print(f"Enter {n} elements:")
for _ in range(n):
  A.append(int(input()))
quicksort(A, 0, n - 1)
print("Sorted elements:")
for num in A:
  print(num, end=' ')
```

Output:

Enter size of the array: 5

Enter 5 elements: 3 8 0 1 2

Sorted elements: 0 1 2 3 8

Complexity of Quick Sort

The time to sort the file is equal to

The time to sort the left partition with i elements, plus

The time to sort the right partition with N–i–1 element

The time to build the partitions

Worst Case Analysis

The pivot is the smallest element

$$T(N) = T(N-1) + cN, N > 1$$
$$T(N-1) = T(N-2) + c(N-1)$$
$$T(N-2) = T(N-3) + c(N-2)$$
$$T(N-3) = T(N-4) + c(N-3)$$
$$T(2) = T(1) + c.2$$

Add all equations:

$$T(N) + T(N-1) + T(N-2) + \ldots + T(2) = T(N-1) + T(N-2) + \ldots + T(2) + T(1) + c(N) + c(N-1) + c(N-2) + \ldots + c.2$$
$$T(N) = T(1) + c \text{ times (the sum of 2 thru N)}$$
$$= T(1) + c(N(N+1)/2 - 1) = O(N2)$$

st–Case Analysis

The pivot is in the middle

$$T(N) = 2T(N/2) + cN$$

Divide by N:

$$T(N) / N = T(N/2) / (N/2) + c$$
$$T(N/2) / (N/2) = T(N/4) / (N/4) + c$$
$$T(N/4) / (N/4) = T(N/8) / (N/8) + c$$

.....

$$T(2) / 2 = T(1) / (1) + c$$

Add all equations

$$T(N) / N + T(N/2) / (N/2) + T(N/4) / (N/4) + ... + T(2) / 2$$
$$= (N/2) / (N/2) + T(N/4) / (N/4) + ... + T(1) / (1) + c.logN$$

After crossing the equal terms

$$T(N)/N = T(1) + cLogN$$
$$T(N) = N + NcLogN = O(NlogN)$$

Average case analysis

Similar computations, resulting in T(N) = O(NlogN)

The average value of T(i) is 1/N times the sum of T(0) rough T(N–1)

$$1/N \text{ S } T(j), j = 0 \text{ thru } N–1$$
$$T(N) = 2/N (S T(j)) + cN$$

Multiply by N

$$NT(N) = 2(S T(j)) + cN*N$$

To remove the summation, we rewrite the equation for –1

$$(N–1)T(N–1)=2(S T(j)) + c(N–1)2, j = 0 \text{ thru } N–2$$

and subtract

$$T(N) – (N–1)T(N–1) = 2T(N–1) + 2cN – c$$
$$NT(N) = (N+1)T(N–1) + 2cN$$

Divide by N(N+1)

$$T(N)/(N+1) = T(N–1)/N + 2c/(N+1)$$
$$T(N)/(N+1) = T(N–1)/N + 2c/(N+1)$$
$$T(N–1)/(N) = T(N–2)/(N–1)+ 2c/(N)$$
$$T(N–2)/(N–1) = T(N–3)/(N–2) + 2c/(N–1)$$

....

$$T(2)/3 = T(1)/2 + 2c /3$$

Add the equations and cross equal terms

$$T(N)/(N+1) = T(1)/2 + 2c \text{ S } (1/j), j = 3 \text{ to } N+1$$

The sum S (1/j), j = 3 to N–1, is about LogN

Thus $T(N) = O(NlogN)$

2.14 MERGE SORT [Oct. 22, April 23]

The Merge Sort algorithm is a divide-and-conquer algorithm that sorts an array by first breaking it down into smaller arrays, and then building the array back together the correct way so that it is sorted.

Divide: The algorithm starts with breaking up the array into smaller and smaller pieces until one such sub-array only consists of one element.

Conquer: The algorithm merges the small pieces of the array back together by putting the lowest values first, resulting in a sorted array.

The breaking down and building up of the array to sort the array is done recursively.

In the animation above, each time the bars are pushed down represents a recursive call, splitting the array into smaller pieces. When the bars are lifted up, it means that two sub-arrays have been merged together.

The Merge Sort algorithm can be described like this:

How it Works?

1. Divide the unsorted array into two sub-arrays, half the size of the original.
2. Continue to divide the sub-arrays as long as the current piece of the array has more than one element.
3. Merge two sub-arrays together by always putting the lowest value first.
4. Keep merging until there are no sub-arrays left.

Take a look at the drawing below to see how Merge Sort works from a different perspective. As you can see, the array is split into smaller and smaller pieces until it is merged back together. And as the merging happens, values from each sub-array are compared so that the lowest value comes first.

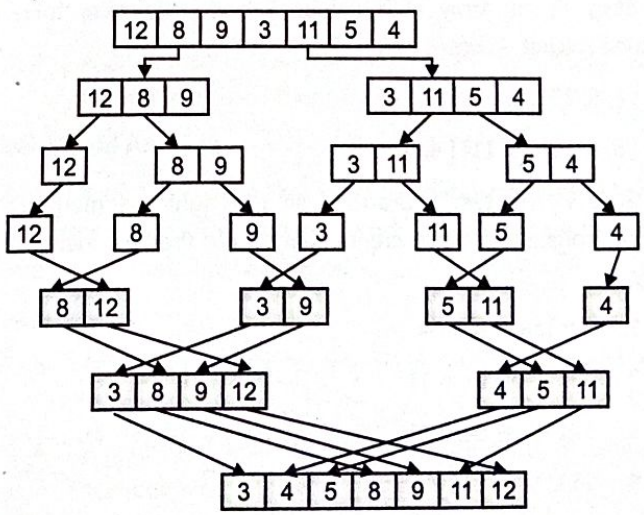


Fig. 2.19: Manual run through

Let's try to do the sorting manually, just to get an even better understanding of how Merge Sort works before actually implementing it in a programming language.

Step 1: We start with an unsorted array, and we know that it splits in half until the sub-arrays only consist of one element. The Merge Sort function calls itself two times, once for each half of the array. That means that the first sub-array will split into the smallest pieces first.

[12, 8, 9, 3, 11, 5, 4]

[12, 8, 9] [3, 11, 5, 4]

[12] [8, 9] [3, 11, 5, 4]

[12] [8] [9] [3, 11, 5, 4]

Step 2: The splitting of the first sub-array is finished, and now it is time to merge. 8 and 9 are the first two elements to be merged. 8 is the lowest value, so that comes before 9 in the first merged sub-array.

[12] [8, 9] [3, 11, 5, 4]

Step 3: The next sub-arrays to be merged is [12] and [8, 9]. Values in both arrays are compared from the start. 8 is lower than 12, so 8 comes first, and 9 is also lower than 12.

[8, 9, 12] [3, 11, 5, 4]

Step 4: Now the second big sub-array is split recursively.

[8, 9, 12] [3, 11, 5, 4]

[8, 9, 12] [3, 11] [5, 4]

[8, 9, 12] [3] [11] [5, 4]

Step 5: 3 and 11 are merged back together in the same order as they are shown because 3 is lower than 11.

[8, 9, 12] [3, 11] [5, 4]

Step 6: Sub-array with values 5 and 4 is split, then merged so that 4 comes before 5.

[8, 9, 12] [3, 11] [5] [4]

[8, 9, 12] [3, 11] [4, 5]

Step 7: The two sub-arrays on the right are merged. Comparisons are done to create elements in the new merged array:

1. 3 is lower than 4
2. 4 is lower than 11
3. 5 is lower than 11
4. 11 is the last remaining value

[8, 9, 12] [3, 4, 5, 11]

Step 8: The two last remaining sub-arrays are me... Let's look at how the comparisons are done in more de... create the new merged and finished sorted array:

3 is lower than 8:

Before [8, 9, 12] [3, 4, 5, 11]

After: [3, 8, 9, 12] [4, 5, 11]

Step 9: 4 is lower than 8:

Before [3, 8, 9, 12] [4, 5, 11]

After: [3, 4, 8, 9, 12] [5, 11]

Step 10: 5 is lower than 8:

Before [3, 4, 8, 9, 12] [5, 11]

After: [3, 4, 5, 8, 9, 12] [11]

Step 11: 8 and 9 are lower than 11:

Before [3, 4, 5, 8, 9, 12] [11]

After: [3, 4, 5, 8, 9, 12] [11]

Step 12: 11 is lower than 12:

Before [3, 4, 5, 8, 9, 12] [11]

After: [3, 4, 5, 8, 9, 11, 12]

The sorting is finished!

CASE STUDY

Social Network Adjacency Matrix Representing Friends Connections Among Millions of Users.

Problem Statement

Social networks like Facebook, Instagram, and Link... must efficiently store and manage friendship connec... among millions (or even billions) of users. The goal i... choose the right data structure for fast access, update, ... scalability of user relationships.

Real-World Context

Every social media platform is essentially a graph of us... where:

- Users are nodes (vertices)
- Friendships are edges (connections)

To manage this, data structures such as adjace... matrices or adjacency lists are used depending on the spa... or density of the graph.

Data Structures Used

Adjacency Matrix (2D Array of size N x N)

matrix[i][j] = 1 means user i is friends with user j

matrix[i][j] = 0 means no friendship exists

Why Use Adjacency Matrix?

- Quick Edge Lookup: O(1) time complexity for checking friendship.
- Easy to implement and visualize.
- Suitable for dense graphs or when relationships are bidirectional and frequently queried.

When to Use Adjacency Matrix over Adjacency List?

Feature	Adjacency Matrix	Adjacency List
Edge lookup time	O(1)	O(k) where k is degree
Space for N users	$O(N^2)$	O(N + E)
Best for	Dense Graphs	Sparse Graphs
Real-world analogy	Facebook mutual friends	Twitter follows

```
lass SocialNetwork:
 def __init__(self, users):
    self.num_users = users
    self.adjacency_matrix = [[False] * users for _ in
ange(users)]
 # Add friendship (bidirectional)
 def add_friendship(self, user1, user2):
    if user1 < self.num_users and user2 < self.num_users:
       self.adjacency_matrix[user1][user2] = True
       self.adjacency_matrix[user2][user1] = True

 # Remove friendship
 def remove_friendship(self, user1, user2):
    self.adjacency_matrix[user1][user2] = False
    self.adjacency_matrix[user2][user1] = False

 # Check if two users are friends
 def are_friends(self, user1, user2):
    return self.adjacency_matrix[user1][user2]

 # Display the adjacency matrix
 def display_matrix(self):
    print("Adjacency Matrix:")
    for row in self.adjacency_matrix:
       print(' '.join(['1' if cell else '0' for cell in row]))

 Main
n = SocialNetwork(5)  # Create a social network with 5 users (0
o 4)
# Add friendships
sn.add_friendship(0, 1)
sn.add_friendship(0, 2)
sn.add_friendship(1, 3)
sn.add_friendship(2, 4)
sn.display_matrix()

print("\nAre user 0 and 1 friends?", "Yes" if sn.are_friends(0, 1)
else "No")
print("Are user 1 and 4 friends?", "Yes" if sn.are_friends(1, 4)
else "No")
```

Complexity Analysis

Operation	Time Complexity
Add/Remove Friendship	O(1)
Check Friendship	O(1)
Display Matrix	$O(N^2)$
Space Usage	$O(N^2)$

Scalability Consideration

For a network of millions of users, an adjacency matrix becomes infeasible due to space:

- For N = 1,000,000, matrix size = 10^{12} bits = 125 GB!
- Use Adjacency List or Compressed Sparse Row (CSR) format for such scenarios.

Enhancements

- Store additional data (timestamp, relationship strength) in a weight matrix.
- Use bitsets instead of bool arrays for memory optimization.
- Apply graph traversal algorithms (BFS/DFS) for finding friends-of-friends.

Real-World Analogy

Think of this matrix like a digital friendship map where every user sits on a row and a column. If two users are friends, there's a 1 in their intersecting cell.

Takeaways

- Adjacency Matrix is optimal for smaller or denser networks.
- It provides instant access to relationship status between any two users.
- For large-scale social networks, hybrid or list-based approaches are preferred.

EXERCISE

1. What is an array in data structures? Explain how an array can be considered an Abstract Data Type (ADT) with examples.
2. List and explain the basic operations that can be performed on arrays. Give a short explanation of each operation (e.g., insertion, deletion, traversal).
3. Explain how arrays are stored in memory. What is meant by row-major and column-major storage in multidimensional arrays?
4. Define 2D and n-dimensional arrays. Write a program or pseudocode to initialize and access a 2D array.
5. What is a sparse matrix? Explain how a sparse matrix can be efficiently represented using 2D arrays. Provide an example.
6. Describe the working of the following searching algorithms:
   (a) Sequential/Linear Search
   (b) Binary Search

(c) Fibonacci Search

(d) Indexed Sequential Search

7. Compare Linear Search and Binary Search in terms time complexity and suitability for unsorted/sort data.

8. Define the concepts of Stability, Efficiency, and Numb of Passes in sorting algorithms. Why are the important for comparing sorting methods?

9. Differentiate between Internal and External Sorting wi examples of when each is used.

10. Write the algorithm and time complexity for any two the following sorting methods:

(a) Bubble Sort

(b) Insertion Sort

(c) Selection Sort

(d) Quick Sort

(e) Merge Sort